

TEMA 1. Lógica y demostraciones

- Proposición: oración declarativa verdadera o falsa, nunca ambas a la vez. $P, q, r, \dots$ denotarán proposiciones. Tienen un valor de verdad (Verdadera o falsa).
- Cálculo proposicional o lógica proposicional: área de la lógica que trata las proposiciones.
- Tabla de verdad: herramienta que relaciona los valores de verdad de proposiciones. Útil para la construcción de proposiciones desde otras e simples.

Operadores lógicos / conectivos lógicos

()

- ↳ Negación ($\neg$) No
- ↳ Conjunción ($\wedge$) y
- ↳ Disyunción / o inclusiva ($\vee$) o
- ↳ o exclusiva ($\oplus$) o una u otra
- ↳ Implicación ($\rightarrow$) Si p entonces q
- ↳ Bicondicional / doble implicación ($\leftrightarrow$) Si y solo si

NOTA: Una asociación que puede ayudar a comprender estos nexos son las puertas lógicas (Álgebra de Boole), estudiado en FC.

Precedencia

• TV ($\neg$) NOT

P	$\neg P$
V	F
F	V

• TV ($\wedge$) AND

P	q	$P \wedge q$
F	F	F
F	V	F
V	F	F
V	V	V

• TV ($\vee$) OR

P	q	$P \vee q$
F	F	F
F	V	V
V	F	V
V	V	V

NOTA: Para que resulte + cómodo:
 $0/1 \equiv F/V$
(variables booleanas)

• TV ($\oplus$) XOR

P	q	$P \oplus q$
F	F	F
F	V	V
V	F	V
V	V	F

• TV ($\rightarrow$)

P	q	$P \rightarrow q$
F	F	V
F	V	V
V	F	F
V	V	V

• TV ($\leftrightarrow$) XNOR

P	q	$P \leftrightarrow q$
F	F	V
F	V	F
V	F	F
V	V	V

• Inversa, Recíproca y Contrarrecíproca (de $p \rightarrow q$)

- ↳ Inversa: $\neg p \rightarrow \neg q$
- ↳ Recíproca: $q \rightarrow p$
- ↳ Contrarrecíproca: $\neg q \rightarrow \neg p$

Ej: Si llueve el equipo A gana.
Inv: Si no llueve, el equipo A no gana.
R: Si el equipo A gana, llueve.
CR: Si el equipo A no gana, no llueve.

Lo más destacable aquí, es que la implicación $\equiv$ contrarrecíproca ($p \rightarrow q \equiv \neg q \rightarrow \neg p$) mientras que las otras 2 son falsas.

• Lenguaje Natural y lenguaje Formal:

El lenguaje natural es ambiguo, lo que se puede evitar trasladando frases a expresiones lógicas, esto se denomina formalizar.

En un argumento matemático: la sustitución, de sentencias por equivalentes, es clave.

EQUIVALENCIAS LÓGICAS

($\equiv$)

• Para que proposiciones o expresiones lógicas sean equivalentes se debe cumplir que sus valores de verdad en todos los casos posibles sean lógicamente equivalentes (igd).

NOTA: para comprobar la equivalencia entre 2 proposiciones o exp. lóg. se usa TV.

FÓRMULAS BÁSICAS

- Tautología: fórmula / exp. lóg. cuyo valor de verdad es siempre V
- Contradicción: fórmula / exp. lóg. que siempre es F.
- Contingencia: fórmula / exp. lóg. que no es ni tautología ni contradicción.

EQUIVALENCIAS LÓGICAS IMPORTANTES

EQUIVALENCIA	NOMBRE
$P \wedge V \equiv P$ $P \vee F \equiv P$	Leyes de identidad
$P \vee V \equiv V$ $P \wedge F \equiv F$	Leyes de dominación
$P \vee P \equiv P$ $P \wedge P \equiv P$	Leyes de idempotencia misma potencia misma variable
$\neg(\neg P) \equiv P$	Ley de la doble negación
$P \vee Q \equiv Q \vee P$ $P \wedge Q \equiv Q \wedge P$	Leyes conmutativas
$(P \vee Q) \vee R \equiv P \vee (Q \vee R)$ $(P \wedge Q) \wedge R \equiv P \wedge (Q \wedge R)$	Leyes asociativas
$P \vee (Q \wedge R) \equiv (P \vee Q) \wedge (P \vee R)$ $P \wedge (Q \vee R) \equiv (P \wedge Q) \vee (P \wedge R)$	Leyes distributivas
$\neg(P \wedge Q) \equiv \neg P \vee \neg Q$ $\neg(P \vee Q) \equiv \neg P \wedge \neg Q$	Leyes de De Morgan
$P \vee (P \wedge Q) \equiv P$ $P \wedge (P \vee Q) \equiv P$	Leyes de absorción
$P \vee \neg P \equiv V(1)$ $P \wedge \neg P \equiv F(0)$	Leyes de negación

• Predicado: propiedad que puede ser verdadera o falsa según el valor de sus variables. Ej: $x > 5$; x : variable / sujeto ; > 5 : predicado. Se escribe como $P(x)$.

↳ Para cada valor de x ; $P(x)$ se convierte en una proposición.

• $P(3)$: " $3 > 5$ " Falso

• $P(7)$: " $7 > 5$ " Verdadero.

• Lógica de Predicados / Lógica de 1er Orden: área de la lógica que trata con Predicados y Cuantificadores.

CUANTIFICADORES

• Cuantificadores: permiten convertir una función proposicional en una proposición completa.

↳ Cuantificador Universal ($\forall$)

$$\forall x P(x)$$

"Para todo x , $P(x)$ es verdadera"

• Si el dominio es $\{x_1, x_2, \dots, x_n\}$:

$$\forall x P(x) \equiv P(x_1) \wedge P(x_2) \wedge \dots \wedge P(x_n)$$

NOTA: Solo es V si todas $P(x_n)$ lo son

↳ Cuantificador Existencial ($\exists$)

$$\exists x P(x)$$

"Existe al menos un x para el que $P(x)$ es verdadera"

• Si el dom es $\{x_1, x_2, \dots, x_n\}$

$$\exists x P(x) = P(x_1) \vee P(x_2) \vee \dots \vee P(x_n)$$

NOTA: Será verdadera si al menos 1 $P(x_n)$ lo es. (No todas han de serlo).

• Cuando un cuantificador se usa sobre x , se dice que x es una **variable ligada**.

Ejemplo: $\forall x P(x, y)$, $x \rightarrow$ ligada a $\forall$, $y \rightarrow$ no (es libre).

• Negación de cuantificadores (Leyes de De Morgan)

↳ Negación del universal: $\neg \forall x P(x) \equiv \exists x \neg P(x)$

↳ Negación del existencial: $\neg \exists x P(x) \equiv \forall x \neg P(x)$

NOTA: parece que sale al cuantificador y lo cambia.

• Cuantificadores anidados.

Cuando tienes 2 cuantificadores = y 2 variables $\neq$ hay que poner como bucles anidados (pp)

Ej: $\forall x \forall y P(x, y)$. El orden no importa

Sin embargo, cuando tienes 2 cuantificadores $\neq$ el orden sí importa $\forall x \exists y P(x, y) \neq \exists y \forall x P(x, y)$.

En matemáticas es clave saber qué métodos se usan para construir argumentos matemáticos.

• Veracidad de Teoremas: teorema es una sentencia (proposición, hecho...).

↳ Demostración: secuencia de sentencias que constituyen un argumento que determina que un teorema es verdadero.

DEMOSTRACIONES

Método deductivo { Para construir demostraciones podemos usar un método que a partir de sentencias ya conocidas (axiomas, postulados, teoremas...) crea nuevas. Las Reglas de Inferencia sirven para enlazar los pasos de una demostración

Conceptos { • Falsas: razonamientos incorrectos
• Lema: más sencillos que un TA

• Corolario: sale de un TA
• Conjeturas: se desconoce valor de V.

• RI

Las RI justifican los pasos dados para
desde una hipótesis $\rightarrow$ conclusión

P
 $P \rightarrow q$
Por lo tanto
Luego q

REGLA DE INFERENCIA

TAUTOLOGÍA

NOMBRE

$\frac{P}{\therefore P \vee q}$	$P \rightarrow (P \vee q)$	Adición
$\frac{P \wedge q}{\therefore P}$	$(P \wedge q) \rightarrow P$	Simplificación
$\frac{P}{q}$ $\therefore P \wedge q$	$((P) \wedge (q)) \rightarrow (P \wedge q)$	Conjunción o ley de combinación
$\frac{P}{P \rightarrow q}$ $\therefore q$	$[P \wedge (P \rightarrow q)] \rightarrow q$	Modus ponens
$\frac{\neg q}{P \rightarrow q}$ $\therefore \neg P$	$[\neg q \wedge (P \rightarrow q)] \rightarrow \neg P$	Modus tollens
$\frac{P \rightarrow q}{q \rightarrow r}$ $\therefore P \rightarrow r$	$[(P \rightarrow q) \wedge (q \rightarrow r)] \rightarrow (P \rightarrow r)$	Silogismo hipotético
$\frac{P \vee q}{\neg P}$ $\therefore q$	$[(P \vee q) \wedge (\neg P)] \rightarrow q$	Silogismo disyuntivo
$\frac{P \vee q}{\neg P \vee r}$ $\therefore q \vee r$	$[(P \vee q) \wedge (\neg P \vee r)] \rightarrow (q \vee r)$	Ley de resolución.

• RI para sentencias cuantificadas.

REGLA DE INFERENCIA	NOMBRE
$\frac{\forall x P(x)}{\therefore P(c)}$	Particularización universal
$\frac{P(c) \text{ arbitrario}}{\therefore \forall x P(x)}$	Generalización universal
$\frac{\exists x P(x)}{\therefore P(c) \text{ para algún elemento } c}$	Particularización existencial
$\frac{P(c) \text{ para algún elemento } c}{\therefore \exists x P(x)}$	Generalización existencial

Nota: un argumento deductivo es correcto si con hipótesis verdaderas la conclusión también lo es.



• Métodos para demostrar teoremas

La mayoría de los teoremas son implicaciones ($\rightarrow$).

↳ Demostraciones directas: la implicación $p \rightarrow q$ se puede demostrar viendo que si p es verdadera q ha de serlo a su vez. Esto es una DD. y se usan RI y TA ya demostradas.

↳ Demostraciones indirectas: $p \rightarrow q$ también se puede demostrar viendo que su **contrarrecíproca**, $\neg q \rightarrow \neg p$, es verdadera, ya que sabemos que son $\equiv$. Esta argumentación es una demostración indirecta.

↳ Demostraciones Vacuas y Triviales: Suponiendo que p es falsa, entonces la implicación $p \rightarrow q$ tiene la forma $F \rightarrow F$ o $F \rightarrow V$ y en cualquier caso es verdadera. Ergo, si se demuestra que p es V entonces se da una **demostración vacua**. La **demostración trivial** se da cuando q es verdadera y se puede demostrar, teniendo una sentencia del tipo $V \rightarrow V$ o $F \rightarrow V$, lo cual es siempre verdadera.

DEMOSTRACIONES

↳ Demostraciones por Reducción al Absurdo: si se puede encontrar una contradicción tal que $\neg p \rightarrow q$ sea V , entonces $\neg p \rightarrow F$, luego $\neg p$ ha de ser F y p , será verdadera.

↳ Demostración por Casos: Se puede demostrar una implicación transformando la misma en una hipótesis como una fórmula combinada de implicaciones que la componen.

C.g. $P \rightarrow q$; $(p_1 \vee p_2 \vee \dots \vee p_n) \rightarrow q$ (podemos demostrar cada una de las n implicaciones).

↳ Demostraciones de Existencia: muchas TA se enuncian haciendo uso de proposiciones y cuantificadores. Afirman la existencia de "algo". Demostrar esto, hay varias formas $\rightarrow$ "Demostraciones de existencia": (DE)

- DE constructiva: encontrar un $a \exists P(a)$ sea V .
- DE no constructiva: no encontramos un $a \exists P(a)$ sea V . Se usa una demostración por reducción al absurdo. Método $\rightarrow$ se muestra que la negación de la cuantificación existencial implica una contradicción.
- Contraejemplos: Lo usamos para demostrar que una sentencia $\forall x P(x)$ es falsa. Es un ejemplo que demuestra el que dicha sentencia es verdadera.
Nota: $\exists$ algunos ejemplos por los que la cuantificación universal es verdadera, esta puede ser falsa.

FALLOS
COMUNES

- $\rightarrow$ Equivocaciones aritméticas o de álgebra básica
- $\rightarrow$ Cada paso en una demostración debe ser correcto.
- $\rightarrow$ La conclusión debe deducirse de los pasos anteriores.

- Se usa para demostrar muchos resultados o T^{as}: para demostrar propiedades, sobre la complejidad de los algoritmos, o teoremas sobre grafos o árboles.

Por tanto, es una técnica para demostrar proposiciones del tipo: $\forall n \in \mathbb{Z}^+, P(n)$ ^{enteros positivos}

PASOS

- Paso Base: se muestra que $P(1)$ es verdadera.
- Paso de inducción: se muestra que $P(k) \rightarrow P(k+1)$ es verdadera $\forall$ entero positivo k .
 $[P(1) \wedge (\forall k P(k) \rightarrow P(k+1))] \rightarrow \forall n P(n)$

La inducción funciona gracias a la **Propiedad del Buen Orden**: si $P(1)$ es V y cada $P(k)$ implica $P(k+1)$ entonces no puede existir un $n \in \mathbb{Z}^+$ donde $P(n)$ sea falsa.

Propiedad del Buen Orden: todo conjunto de enteros no negativos tiene un elemento mínimo.

TEMA 2: Conjuntos y Relaciones.

- Conjuntos: estructura discreta sobre la que se construyen todas las que estudiaremos (relaciones, combinaciones, grafos, etc.). Es una colección desordenada de objetos.

- Igualdad de conjuntos: 2 conjuntos son $= \iff$ contienen los mismos elementos.

- Descripción de un conjunto: existen muchas maneras de nombrar los objetos que contiene un conjunto. Dando el nombre del conjunto, enumerando los elementos $\{a, e, i, o, u\}, \dots$

La notación por comprensión: se caracterizan todos los elementos del conjunto dando la(s) propiedad(es) que estos tienen. Esta se usa a veces es imposible enumerar todos los miembros del conjunto.

$$O = \{x \mid x \text{ es un entero positivo } < 10\}$$

- Diagramas de Venn: representación gráfica de conjuntos.

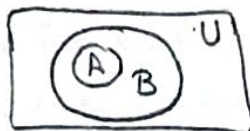
→ Conjunto universal $\{U\}$: que contiene todos los objetos posibles, viene dado por un rectángulo. Dentro de este, los conjuntos se representan por círculos u otras formas geométricas.



- Conceptos y notaciones de conjuntos
- Pertenencia ($\in$): cuando un elemento pertenece a un conjunto; $a \in V$; $b \notin V$
 - Conjunto vacío ($\emptyset$): (o Conjunto nulo) es un conjunto sin ningún elemento.
- Nota:** no confundir el conjunto $\emptyset$ con el conjunto unitario $\{ \emptyset \}$, donde el único elemento es el conjunto vacío.

- Subconjunto: el conjunto A se dice que es un subconjunto de B si, y solo si, todo elemento de $A \in B$. $A \subseteq B \Leftrightarrow \forall x (x \in A \rightarrow x \in B)$.

Lo si $A \neq B$ entonces $A \subset B$ y se dice que A es un subconjunto propio de B.



TEOREMA: todo conjunto no vacío, S, tiene al menos, dos subconjuntos, el $\emptyset$ y el mismo.

INTRODUCCIÓN A LA TEORÍA DE CONJUNTOS

- Cardinal: en problemas de recuento muchas veces necesitamos el tamaño de conjuntos, es decir, el nº de elementos a lo cual se le denomina cardinal ($|S|$). Un conjunto es infinito si no es finito.

- Conjunto de Partes de un conjunto / Particiones de un conjunto: Dado un conjunto S, el conjunto de las partes de S es el conjunto de todos los subconjuntos de S. Su notación es $P(S)$. Si un conjunto tiene n elementos entonces el conjunto de sus partes tiene 2^n .

E.g. $W = \{a, e, i\}$

$P(W) = \{\emptyset, \{a\}, \{e\}, \{i\}, \{a, e\}, \{a, i\}, \{e, i\}, \{a, e, i\}\}$

$$\begin{aligned} P(\emptyset) &= \{\emptyset\} \\ P(\{a\}) &= \{\emptyset, \{a\}\} \end{aligned}$$

Producto Cartesiano

- Colección ordenada: los elementos que pertenecen a un conjunto no están ordenados. El orden en una colección puede ser importante.

Lo Estructura: n -tuplas ordenadas $(a_1, a_2, \dots, a_n)$. Dos n -tuplas se dicen que son $=$ $\Leftrightarrow$ cada par correspondiente de sus elementos lo son:

$$(a_1, a_2, \dots, a_n) = (b_1, b_2, \dots, b_n) \Leftrightarrow a_i = b_i ; i = 1, 2, \dots, n.$$

Nota: en particular las 2-tuplas se llaman "pares ordenados"

El producto cartesiano de 2 conjuntos, A y B ($A \times B$), es el conjunto de pares ordenados (a, b) , donde $a \in A$ y $b \in B$. $A \times B = \{(a, b) | a \in A \wedge b \in B\}$

- Relación (R): una relación R de A en B es un subconjunto del producto cartesiano; $R \subseteq A \times B$. $R = \{(a, b) | a \in A \wedge b \in B\}$

E.g: $A = \{a, b, c\}$; $B = \{0, 1, 2, 3\}$

$$R = \{(a, 0), (a, 1), (a, 2), (a, 3), (b, 1), (b, 2), (c, 1), (c, 3)\}$$

- Producto cartesiano de n conjuntos. Para n conjuntos, el resultado son n -tuplas.

$A_1 = \{x\}$ $A_2 = \{1, 2\}$ $A_3 = \{p, q\}$

$$A_1 \times A_2 \times A_3 = \{(x, 1, p), (x, 1, q), (x, 2, p), (x, 2, q)\}$$

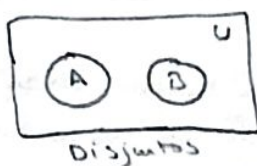
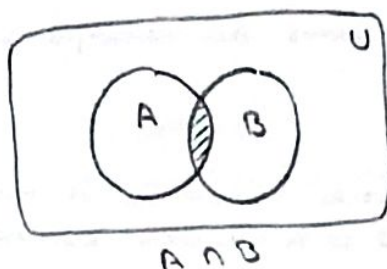
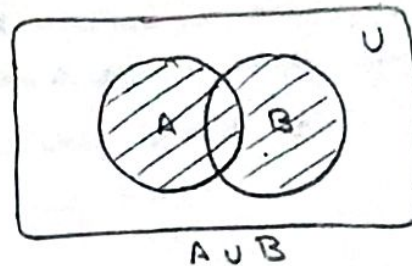
NOTA: RECUERDA!, pares ordenados son iguales, $(a, b) = (c, d) \Leftrightarrow a = c$ y $b = d$.

El orden es importante $(a, b) \neq (b, a)$ si $a \neq b$.

• Unión de Conjuntos ($A \cup B$)

Es el conjunto cuyos elementos son elementos tanto de A, como de B, como de ambos.

$$A \cup B = \{x | x \in A \vee x \in B\}$$



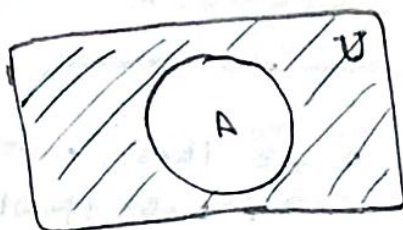
¡Ojo! podemos pensar que el cardinal de la unión es $|A \cup B| = |A| + |B|$. Esto es un error pues se cuenta x2 los elementos de la intersección.

• Diferencia de Conjuntos ($A - B$)

Conjunto que contiene los elementos que solo se encuentran en uno de los conjuntos.

$$A - B = \{x | x \in A \wedge x \notin B\}$$

$$A - B = A \cap \overline{B} \text{ es el complementario de } B.$$



• Intersección de Conjuntos ($A \cap B$)

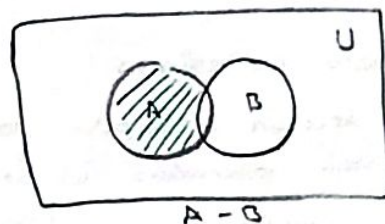
Conjunto cuyos elementos son aquellos comunes tanto a A como B.

$$A \cap B = \{x | x \in A \wedge x \in B\}$$

Nota: 2 conjuntos son disjuntos si su intersección es el $\emptyset$.

La Pto de inclusión exclusiva: el cardinal de la intersección de conjuntos es útil para calcular el de la unión.

$$|A \cup B| = |A| + |B| - |A \cap B|$$



• Conjunto Complementario ($\bar{A}$)

El conjunto complementario de A, $\bar{A}$, es el complementario de A con respecto a U. O sea el complementario del conjunto A es $U - A$.

$$\bar{A} = \{x | x \notin A\}$$

IDENTIDAD

NOMBRES

$A \cup \emptyset = A$ $A \cap U = A$	Leyes de identidad
$A \cup U = U$ $A \cap \emptyset = \emptyset$	Leyes de dominación
$A \cup A = A$ $A \cap A = A$	Leyes idempotentes
$\overline{\overline{A}} = A$	Ley de complementación
$A \cup B = B \cup A$ $A \cap B = B \cap A$	Leyes conmutativas

IDENTIDAD

NOMBRES

$A \cup (B \cap C) = (A \cup B) \cap C$ $A \cap (B \cup C) = (A \cap B) \cup C$	Leyes asociativas
$A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$ $A \cup (B \cap C) = (A \cup B) \cap (A \cup C)$	Leyes distributivas
$\overline{A \cup B} = \bar{A} \cap \bar{B}$ $\overline{A \cap B} = \bar{A} \cup \bar{B}$	Leyes de De Morgan
$A \cup (A \cap B) = A$ $A \cap (A \cup B) = A$	Leyes de absorción
$A \cup \bar{A} = U$ $A \cap A = \emptyset$	Leyes de complemento

IDENTIDADES
DE
CONJUNTOS

• Igualdad entre conjuntos por subconjunto.

Dos conjuntos tmb son iguales si se demuestra que un conjunto es subconjunto de otro y viceversa. C.g. por demostración de ley de De Morgan.

↳ 2ª ley de De Morgan: $\overline{A \cap B} = \bar{A} \cup \bar{B}$. Si $x \in \overline{A \cap B}$; por definición de la $\cap$, $\neg((x \in A) \wedge (x \in B))$ es V. Aplicando leyes de De Morgan $\rightarrow \neg(x \in A) \vee \neg(x \in B)$, por definición de $\neg \rightarrow x \notin A \vee x \notin B$ y aplicando complementario $x \in \bar{A} \cup \bar{B}$.

• Tablas de pertenencia

Las identidades entre conjuntos se puede demostrar, tmb, usando tablas de pertenencia. Si un elemento pertenece a un conjunto se usa 1, si no, se usa 0.

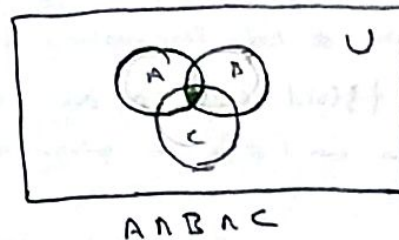
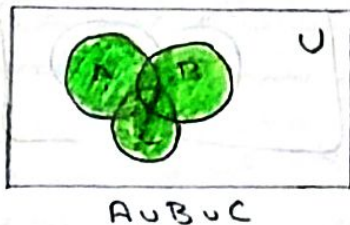
C.g. Propiedad distributiva $\rightarrow \boxed{A \cap (B \cup C) = (A \cap B) \cup (A \cap C)}$

A	B	C	$B \cup C$	$A \cap (B \cup C)$	$A \cap B$	$A \cap C$	$(A \cap B) \cup (A \cap C)$
0	0	0	0	0	0	0	0
0	0	1	1	0	0	0	0
0	1	0	1	0	0	0	0
0	1	1	1	0	0	0	0
1	0	0	0	0	0	0	0
1	0	1	1	1	0	1	1
1	1	0	1	1	1	0	1
1	1	1	1	1	1	1	1

↑ coinciden $\rightarrow =$ ↑

• Uniones e Intersecciones Generalizadas.

Como la $\cup$ y la $\cap$ de conjuntos satisfacen la ley asociativa:



↳ Se puede tmb considerar uniones e intersecciones de un n° arbitrarios de conjuntos.

↳ La unión de una colección de conjuntos es el conjunto que contiene aquellos elementos que como mín pertenecen a un conjunto de la colección: $A_1 \cup A_2 \cup \dots \cup A_n = \bigcup_{i=1}^n A_i$

↳ Para la intersección con una colección de conjuntos, es el conjunto cuyo elementos son miembros de todos los conjuntos de la colección: $A_1 \cap A_2 \cap \dots \cap A_n = \bigcap_{i=1}^n A_i$

Para almacenar conjuntos hemos de almacenar los elementos del susodicho ordenadamente, lo cual simplifica el cálculo de operaciones entre conjuntos. Para ello ordenamos primero los elementos del conjunto universal U finito y cuyos n° de elementos no sea mayor que la memoria.

$$U = \{a_1, a_2, a_3, \dots, a_n\}$$

Luego representamos un subconjunto A de U mediante la cadena de bits de longitud n en la que el bit i -ésimo es 1 si a_i pertenece a A y 0 si no.

→ Complementario de A : la cadena inversa, el $\bar{A}$, es dar cambiando 1 a 0 y viceversa.

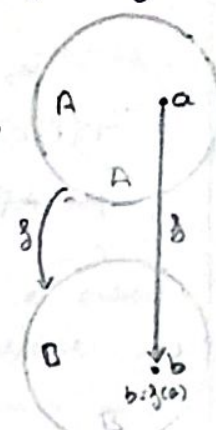
→ $\cup$ de conjuntos: se obtiene de la operación booleana OR entre cadenas de bits.

→ $\cap$ de conjuntos: con operación booleana AND.

• Función: Sean A y B conjuntos. Una función f de A en B es una asignación de exactamente un elemento de B a cada elemento de A ; $f(a) = b$ si " b " es el único elemento de B asignado por la función f al elemento " a " de A ; $f: A \rightarrow B$

• A es el **dominio** de f y B el **codominio** de f .

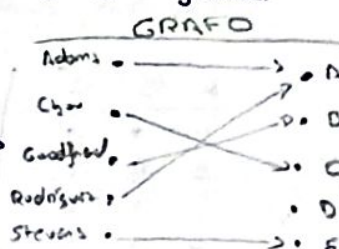
Si $f(a) = b$, b es la **imagen** de a y a es una **preimagen** de b . El **rango o imagen de f** es el conjunto de todas las imágenes de elementos de A . Tamb se llama, función f de A en B o f transforma A en B .



• Especificar funciones: las funciones se pueden especificar de varias formas.

• Declaración explícita de la f : Dominio de G

Adams, Chau, Goodfriend, Rodríguez, Stevens f, el codominio es $\{A, B, C, D, E\}$.



• Fórmulas: $f(x) = a_0 + \sum_{n=1}^{\infty} \left(a_n \cos \frac{n\pi x}{L} + b_n \sin \frac{n\pi x}{L} \right)$

• Suma / producto: funciones de valores reales, con el mismo dominio, se pueden sumar y multiplicar Cg. $(f_1 + f_2)(x) = f_1(x) + f_2(x)$; $(f_1 \cdot f_2)(x) = f_1(x) \cdot f_2(x)$

• Si tenemos un subconjunto $S \subseteq A$ y una función $f: A \rightarrow B$. La imagen de S es el conjunto de todos los valores que toma f al aplicar la función a los elementos de S . $f(S) = \{f(s) \mid s \in S\}$, es decir, cogemos todas las imágenes de los elementos de S y formamos con ellas un subconjunto de B .

• Funciones Inyectivas, Sobreyectivas y Biyectivas

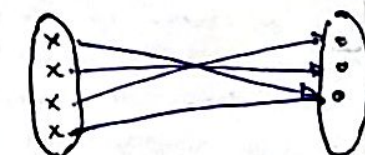
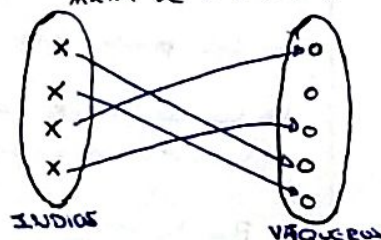
• Inyectivas: una f es inyectiva $\Leftrightarrow f(x) = f(y) \Rightarrow x = y$, es decir, $\forall x, y (f(x) = f(y) \Rightarrow x = y)$, es decir, que los elementos del codominio solo pueden ser imagen de 1 valor del dominio.

• Sobreyectivas: $\Leftrightarrow \forall b \in B \exists a \in A \mid f(a) = b$.

Por tanto, es sobreyectiva si todo valor del codominio es imagen de algún valor del dominio. (Puede ser de + de 1, no como inyectivas)

• Biyectiva: $\Leftrightarrow$ es inyectiva y sobreyectiva

Vaqueros con solo 1 flecha mata de 1 a un vaquero.



NO sobrevive nadie, todos reciben con min un flechazo.

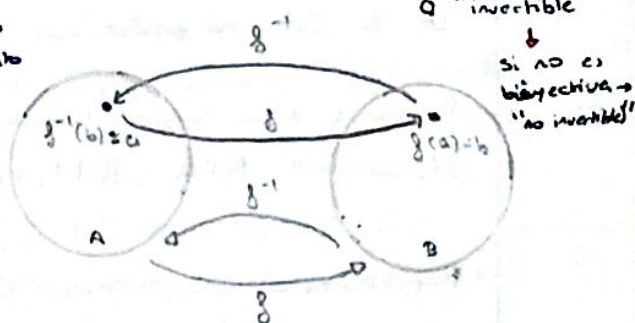
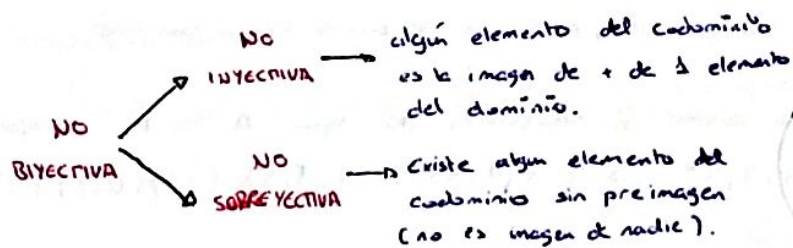


Todos se llevan un flechazo, pero solo hay un objetivo único por lado

NOTA: una f cuyo dom y Codom $\in \mathbb{R}$ es estrictamente creciente si $f(x) < f(y)$ si $x < y$. Estrictamente decreciente si $f(x) > f(y)$ si $x < y$. Para las funciones cuya imagen y codominio son $\mathbb{R}$, estas son sobreyectivas.

• Funciones Inversas y Composición de Funciones.

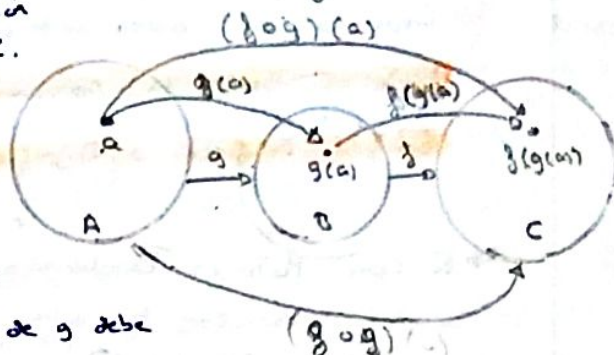
La función inversa: para que una función tenga inversa ha de ser **biyectiva**.



La Composición de funciones: sea g una función del conjunto A al conjunto B y f otra de B a C.

La composición de las funciones f y g sería:

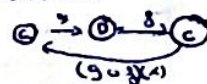
$$(f \circ g)(x) = f(g(x))$$



Básicamente metes una función dentro de otra la cual achá ahora como la incógnita de la función.

• Para que pueda haber composición, la imagen de g debe ser un subconjunto del dominio de f .

• La propiedad conmutativa **NO** se cumple: $(f \circ g)(x) \neq (g \circ f)(x)$ — sería el el salida contrario.



• Gráfica de Funciones

A una función $f: A \rightarrow B$ se le puede asignar un conjunto de pares de $A \times B$. Este conjunto de pares ordenados es la gráfica de la función. $\{(a, b) | a \in A \text{ y } f(a) = b\}$

• Funciones Parte Entera

La función parte entera: asigna a un número real x el mayor entero que es menor o igual que x , $\lfloor x \rfloor$ o $[x]$ ($\leq$)

La función parte entera por exceso: asigna a un número real x el menor entero mayor o igual que x , se denota por $\lceil x \rceil$. ($\geq$)

Ej.

$$\lfloor \frac{1}{2} \rfloor = 0, \lceil \frac{1}{2} \rceil = 1, \lfloor -\frac{1}{2} \rfloor = -1, \lceil -\frac{1}{2} \rceil = 0, \lfloor 7 \rfloor = 7, \lceil 7 \rceil = 7$$

NOTA: $\lfloor -x \rfloor = -\lceil x \rceil, \lceil -x \rceil = -\lfloor x \rfloor$

• Relación

Sea A y B 2 conjuntos. Una relación binaria de A en B es un conjunto R de pares ordenados en los que el primer elemento de cada par ordenado es un elemento de A y el segundo un elemento de B. (o Producto cartesiano.) $a R b \rightarrow (a, b) \in R$

Ej. $A = \{0, 1, 2\}, B = \{a, b\}$

$$R = \{(0, a), (0, b), (1, a), (1, b), (2, a), (2, b)\}$$

• $\exists$ relaciones n-arias, que expresan relaciones entre elementos de + de 2 conjuntos.

• Las relaciones son una generalización de las funciones, pudiendo expresar correspondencias + amplias entre conjuntos.

- En base al punto anterior, las relaciones pueden ser **multívocas**, es decir que un elemento de A puede estar relacionado con + de uno de B. Esto no pasaba con las funciones, pues solo se puede tener una imagen.

- Relaciones de un conjunto consigo mismo: 2 relaciones del tipo A en A, subconjunto del conjunto $A \times A$. $A = \{1, 2, 3, 4\} \rightarrow R = \{(1, 1), (1, 2), (1, 3), (1, 4), (2, 2), (2, 3), \dots\}$

- Representación de relaciones:

↳ Enumerar sus pares ordenados (Ya estudiado).

↳ **Matrices Booleanas**. Apropriadadas para informática (con 0s y 1s).

↳ **Grafos Dirigidos o Digrafos** $\rightarrow$ para atender las propiedades de R.

↳ R con Matrices Booleanas (HB)

Sea una relación R entre los conjuntos ordenados A y B, cada entrada (m_{ij}) vale 1 si $(a_i, b_j) \in R$ y 0 si $(a_i, b_j) \notin R$. La matriz depende del orden elegido para A y B.

E.g: $A = \{1, 2, 3\}$, $B = \{1, 2\}$, $R = \{(a, b) \mid a > b\}$

$R = \{(2, 1), (3, 1), (3, 2)\}$

MATRIX

$$\begin{matrix} & \begin{matrix} b_1 & b_2 \end{matrix} \\ \begin{matrix} a_1 \\ a_2 \\ a_3 \end{matrix} & \begin{pmatrix} 0 & 0 \\ 1 & 0 \\ 1 & 1 \end{pmatrix} \end{matrix}$$

• Propiedades (solo si $\square$)

↳ Reflexiva: solo si los elementos de la diagonal cumplen $m_{ii} = 1$ $\begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$
por tanto su HB será una matriz diagonal.

↳ Simétrica: solo si $m_{ij} = m_{ji}$ para todos los i, j . Es simétrica si $M_R = M_R^t$,

↳ Antisimétrica: si $m_{ij} = 1$ con $i \neq j$, entonces $m_{ji} = 0$, es decir, m_{ji} ha de ser el contrario de m_{ij} .

• Combinación y Representación de Relaciones

↳ Combinación de Relaciones: son = que entre conjuntos. Se pueden usar las operaciones $\cup$ e $\cap$ en matrices booleanas para ver las matrices que representan esas combinaciones de Rs.

E.g:

$$M_{R_1} = \begin{pmatrix} 1 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix}$$

$$M_{R_2} = \begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 0 & 0 \end{pmatrix}$$

$$M_{R_1 \cup R_2} = \begin{pmatrix} 1 & 0 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 0 \end{pmatrix}$$

$$M_{R_1 \cap R_2} = \begin{pmatrix} 1 & 0 & 1 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix}$$

↳ Matriz de composición de relaciones: Si R es una relación de A en B, S una de B en C. A, B, C son m, n y p elementos respectivamente, tenemos $S \circ R \Rightarrow M_{S \circ R}$

$M_{SOR} = [t_{ij}]$, $M_R = [r_{ij}]$, $M_S = [s_{ij}]$ matrices booleanas de dimensión $m \times p$, $m \times n$, $n \times p$, que representen $S \circ R$, R y S respectivamente.

↳ El par ordenado $(a_i, c_j) \in S \circ R \iff \exists b_k$ tal que $(a_i, b_k) \in R$ y $(b_k, c_j) \in S$

↳ $t_{ij} = 1 \iff r_{ik} = s_{kj} = 1$ para algún "k". Por definición del producto booleano esto es: $M_{SOR} = M_R \odot M_S$. (o sea multiplicarlas) ^{NOTA: usando AND y OR}

E.g.:

RELACIONES

$$M_R = \begin{pmatrix} 1 & 0 & 1 \\ 1 & 1 & 0 \\ 0 & 0 & 0 \end{pmatrix}, \quad M_S = \begin{pmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 1 \end{pmatrix} \quad M_{SOR} = M_R \odot M_S = \begin{pmatrix} 1 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 0 \end{pmatrix}$$

• Representación de Relaciones con Grafos Dirigidos

Ya hemos visto grafos. En este caso, los elementos de los conjuntos son puntos y cada par ordenado se representa con una arista orientada.

Un digrafo / grafo dirigido tiene un conjunto de vértices / nodos y aristas formados por ordenados del estilo (a, b) , siendo a el vértice inicial y b el final. La arista (a, a) es llamada **bucle**.

• Propiedades:

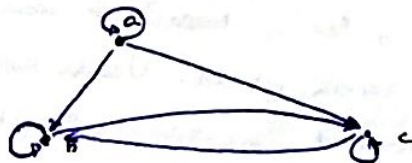
↳ Reflexiva: $\iff$ hay un bucle en cada nodo del digrafo $\forall (x, x) \in R$.

↳ Simétrica: $\iff$ cada vez que hay una arista entre vértices distintos del digrafo, (x, y) , existe una arista en sentido opuesto (y, x) .

Luego ambos pares están en R

↳ Antisimétrica: $\iff$ no existe ninguna pareja de aristas en sentidos opuestos uniendo 2 vértices diferentes del digrafo.

↳ Transitiva: $\iff$ Siempre que hay una arista de un nodo x a uno y , y otra que va del y al z , hay una tercera que une el vértice x con el z .



• Relaciones de Equivalencia.

Una relación R en un conjunto A es una relación de equivalencia (RE) si R es reflexiva, simétrica y transitiva. 2 elementos relacionados por una RE son equivalentes, además cualquier elemento de esa RE equivale a sí mismo y si $a \equiv b$ y $b \equiv c \rightarrow a \equiv c$.

↳ clases de equivalencia si R es un RE en un conjunto A el conjunto de todos los elementos relacionados con un elemento a de A se llama clase de equivalencia de a $[a]_R = \{s | (a, s) \in R\}$

* Si $b \in [a]_R \rightarrow b$ es un representante de dicha clase de equivalencia

Una de las RE + usadas es la "Congruencia módulo m ", donde $m \in \mathbb{Z}^+$ y $m > 1$.

RELACIONES

- Definición: a y b enteros y m entero positivo, entonces a es congruente con b módulo m si m divide a $a - b$. Notación: $a \equiv b \pmod{m}$

• ARITMÉTICA MODULAR

La AM nos permite representar y manejar situaciones en la que nos interesa el "resto" de la división por un entero.

→ a es congruente con b módulo m , $a \equiv b \pmod{m}$ donde b es el resto de a/m .

Ej: Reloj de 24h. Dentro de 45 horas qué hora será?

La Sol: El resto de $(45 + \text{hora actual}) / 24 \rightarrow ((45 + 12) / 24) \rightarrow 9 \text{ horas}$

Teoremas:

- Sea m un $n^\circ \mathbb{Z}^+$, a y b son congruentes módulo $m \Leftrightarrow \exists k \mid a = b + Km$.
- Sea m un $n^\circ \mathbb{Z}^+$. Si $a \equiv b \pmod{m}$ $c \equiv d \pmod{m} \rightarrow a + c \equiv b + d \pmod{m}$ y $ac \equiv bd \pmod{m}$.

TEMA 3: Combinatoria y Probabilidad Discreta.

- Combinatoria: el estudio de cuantas posibles agrupaciones de objetos. La técnica de saber cuantos elementos hay en un colectivo sin tener que contarlos.

- La Regla del Producto (RP): se aplica cuando una tarea se compone de diferentes partes. Si hay una tarea que se puede dividir en 2 sub-tareas consecutivas, y hay n_1 formas de realizar la primera y n_2 de realizar la segunda tras la primera, entonces hay $n_1 \cdot n_2$ formas de completar la tarea. Esto se puede extender a más tareas.

Ej: En una sala hay 32 ordenadores. Cada ordenador tiene 28 puertos. ¿Cuántos puertos hay?

RP: $32 \cdot 28 = 896$ puertos distintos.

- La Regla de la Suma: Si una tarea se puede hacer de n_1 maneras y una segunda tarea de n_2 maneras y las 2 tareas son incompatibles, entonces hay $n_1 + n_2$ formas de realizar la tarea global. Usada cuando una tarea se puede resolver eligiendo una entre una serie de sub-tareas alternativas. Tmb se puede estudiar

Ej: hay que elegir un alumno de entre 3 cursos. $2^\circ \rightarrow 20$, $3^\circ \rightarrow 17$ y $4^\circ \rightarrow 9$ ¿cuántas formas hay de escoger a ese representante?

RS: $20 + 17 + 9 = 46$ formas diferentes de elegir al alumno.

- No obstante, $\exists$ muchos problemas irresolubles aplicando solo RP o RS, pues requieren la aplicación simultánea de ambos.

Ej: Contraseña con longitud 6-8 caracteres, con dígitos de (0-9) o letra mayúscula (26), min 1 d. s/h.

36 36 36 36 36 36

$26^6 - 26^6$ (solo letras) = 16

$R = P_6 + P_7 + P_8$
 $L_6 + L_7 + L_8$

PRINCIPIOS
BÁSICOS
DE
CONTEO

PRINCIPIO DE INCLUSIÓN EXCLUSIÓN

Principio de inclusión-exclusión.

Si 2 tareas pueden realizarse o suceder simultáneamente, no puede usarse la RS para contar las maneras en que se puede realizar una de las 2 tareas, ya que estaríamos contando 2 veces las que se pueden realizar a la vez.

Para evitar este error, se suman las maneras de realizar cada una de ellas y se restan las formas de realizar las 2 tareas simultáneamente. Este Pmo.

Se puede reformular en términos de conjuntos sea A_1 y A_2 conjuntos y T_1 y T_2 tareas de escoger un elemento de cada conjunto. Hay $|A_1|$ formas de hacer T_1 y $|A_2|$ de hacer T_2 . Como hay $|A_1 \cup A_2|$ formas de hacer $T_1 \cup T_2$:

$$|A_1 \cup A_2| = |A_1| + |A_2| - |A_1 \cap A_2|$$

E.g: ¿Cuántas cadenas de 8 bits podemos formar que comiencen bien con un 1 o terminen con 00?

→ Cadenas que empiecen con 1: 2^7

→ Cadenas que terminen 00: 2^6

→ Cadenas de la forma 1...00: 2^5

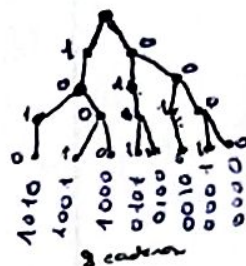
→ Sol: $2^7 + 2^6 - 2^5 = 160$

Algunos problemas de conteo se pueden resolver haciendo uso de diagramas en árbol.

DIAGRAMAS EN ÁRBOL

Los diagramas se generan a partir de una raíz de la que parten ramas con las posibles elecciones a ese nivel. Los resultados posibles están representados por hojas (extremos de las ramas). El valor final se corresponde con los itinerarios posibles desde un nodo hoja hasta la raíz.

¿Cuántas cadenas de 4 bits no tiene 2 de consecutivos?



PRINCIPIO DEL PALOMAR O DIRICHLET

• Principio del Palomar: "si con $K+1$ o más elementos se forman K agrupamientos, existirá al menos 1 agrupamiento que contendrá 2 o + elementos".

E.g: Si tenemos 12 palomeros y 13 palomas:

Al menos un palomar tendrá 2 o + palomas.

• Principio del Palomar Generalizado: Teorema: si se colocan N objetos en K cajas, existe al menos 1 caja con al menos $\lceil N/K \rceil$ objetos.

La Demostración: Si ninguna de las cajas contiene + de $\lceil N/K \rceil - 1$ objetos. Luego,

el número total de objetos será a lo más: $K(\lceil N/K \rceil - 1) < K((N/K) + 1 - 1) = N$.

usando la desigualdad: $\lceil N/K \rceil < (N/K) + 1$. Esto contradice el hecho de que a total hay N objetos.

• En muchas aplicaciones del Principio del Palomar, hay que cuidar la manera en que se eligen los objetos que se ponen en las cajas, necesitando ser elegidos de manera inteligente.

TEOREMA: Toda sucesión de n^2+1 números R distintos, contiene una subsucesión de longitud $n+1$ que es bien estrictamente creciente o estrictamente decreciente.

NOTA: El principio del Palomar Generalizado se puede aplicar en una rama importante de la combinatoria: la teoría de Ramsey. Se basa en la distribución de los subconjuntos de elementos de un conjunto.

• Permutaciones

Una permutación de un conjunto de n elementos es una ordenación concreta de todos los elementos. Ergo, **importa el orden** y **admite reemplazo**.

↳ r -permutaciones

$$P(n, r) = n(n-1)(n-2)\dots(n-r+1) = \frac{n!}{(n-r)!}$$

↳ Población
↳ n° de muestras

PERMUTACIONES

COMBINACIONES

NOTA: Cuando $n=r \rightarrow P(n) = n! ; P(n) = P(n, n)$

↳ Con reemplazo

$$P(n, r) = n^r$$

• Combinaciones

Similar a la permutación con la diferencia de ser una selección no ordenada. Luego, **NO importa el orden** y se **admite reemplazo**. Se denota por $C(n, r)$ y por $\binom{n}{r}$ = coeficiente binomial

↳ r -combinaciones

$$C(n, r) = \frac{n!}{r!(n-r)!}$$

↳ Porque el orden no importa $\{a, b\} = \{b, a\}$

↳ Con reemplazo

$$C(n, r) = \frac{(n+r-1)!}{r!(n-1)!}$$

↳ grupos o
casos en los que divide
↳ n° de muestras

• Permutaciones con objetos indistinguibles.

TEOREMA: El n° de permutaciones diferentes de n objetos donde hay n_1 objetos indistinguibles de tipo 1; n_2 objetos indistinguibles de tipo 2, ..., n_k objetos indistinguibles del tipo k , es: $\frac{n!}{n_1! n_2! \dots n_k!}$

Ej: Palabra SALSA. Formas de construir palabras diferentes:
 $n=5, n_1=2, n_2=2, n_3=1$

$$\frac{5!}{2! 2! 1!} = \frac{120}{4} = 30 \rightarrow 30 \text{ palabras distintas.}$$

→ Monomio: expresión algebraica elemental $(a) (2b) (3c/2)$

→ Binomio: suma o resta de 2 monomios o suma de 2 elementos $(a+b) (a+2b)$

→ Polinomio: interviene las operaciones de suma, resta y producto $(a+2b-3c)$

• Teorema del binomio: es una ecuación usada para desarrollar expresiones de la forma $(a+b)^n$ para algún número natural n . Además nos permite conocer que coeficiente acompaña a un término dado por $a^k b^{n-k}$.

$$(a+b)^n = \sum_{k=0}^n \binom{n}{k} a^{n-k} b^k \quad \left[\begin{array}{l} \text{Coeficiente combinatorio} \\ (a+b)^4 = \binom{4}{0}a^4 + \binom{4}{1}a^3b + \binom{4}{2}a^2b^2 + \dots \end{array} \right]$$

TEOREMA

DEL

BINOMIO

- Corolario 1: para valores de n que sean enteros no negativos.

$$\sum_{k=0}^n \binom{n}{k} = 2^n \quad \text{DEMOSTRACIÓN: } 2^n = (1+1)^n = \sum_{k=0}^n \binom{n}{k} 1^k 1^{n-k} = \sum_{k=0}^n \binom{n}{k}$$

TEOREMA DEL BINOMIO

- Corolario 2: sea n un entero positivo $\rightarrow$

$$\sum_{k=0}^n (-1)^k \binom{n}{k} = 0$$

$$\text{DEMOSTRACIÓN: } 0 = 0^n = (-1+1)^n = \sum_{k=0}^n \binom{n}{k} (-1)^k 1^{n-k} = \sum_{k=0}^n \binom{n}{k} (-1)^k$$

- Corolario 3: sea n un entero positivo. Entonces

$$3^n = \sum_{k=0}^n \binom{n}{k} 2^k$$

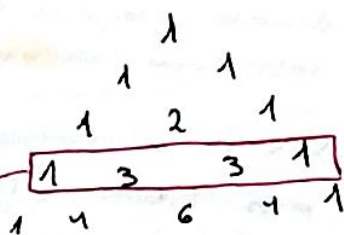
- El Triángulo de Pascal y la Identidad de Pascal.

Identidad de Pascal: sean n y k enteros $+$, tal que $n > k \rightarrow \binom{n+1}{k} = \binom{n}{k-1} + \binom{n}{k}$

Demostación: T , conjunto de $n+1$ elementos donde $a \in T$, $S = T - \{a\}$. Puede haber un subconjunto $\binom{n+1}{k}$ de k elementos en T . Este subconjunto puede contener a a o no. Además como $S \subseteq \binom{n}{k-1}$ de $k-1$ elementos, hay los mismos subconjuntos de T que contienen a a y $\binom{n}{k}$ subconjuntos que no contienen a a .

Triángulo de Pascal.

Para entender el triángulo de Pascal $\binom{n}{0} = 1 = \binom{n}{n}$



Nota: súper útil para nos muestra los coeficientes que se genera $a^n - n a^{n-1} b + \dots$

$$\begin{pmatrix} 0 \\ 0 \end{pmatrix} \quad \begin{pmatrix} 1 \\ 0 \end{pmatrix} \quad \begin{pmatrix} 1 \\ 1 \end{pmatrix} \\ \begin{pmatrix} 2 \\ 0 \end{pmatrix} \quad \begin{pmatrix} 2 \\ 1 \end{pmatrix} \quad \begin{pmatrix} 2 \\ 2 \end{pmatrix} \\ \begin{pmatrix} 3 \\ 0 \end{pmatrix} \quad \begin{pmatrix} 3 \\ 1 \end{pmatrix} \quad \begin{pmatrix} 3 \\ 2 \end{pmatrix} \quad \begin{pmatrix} 3 \\ 3 \end{pmatrix}$$

$a^3 + 3a^2b + 3ab^2 + b^3$ (Cada fila son los coeficientes del desarrollo del binomio $(a+b)^n$)

NOTA DE INTERÉS: nace en el S. XVII en juegos de azar.

- Probabilidad finita: La probabilidad de un suceso E , que es un subconjunto de un espacio muestral finito S , de resultados igualmente probables, es $p(E) = \frac{|E|}{|S|}$
E.g: Urna con 5 bolas rojas y 4 azules, cuál es la probabilidad de que una bola escogida al azar sea roja? $p = 5/9$.

Probabilidad de combinaciones de sucesos: Las técnicas de recuento permiten calcular la probabilidad de sucesos definidos en términos de otros.

Teorema: Sea E un suceso de un espacio muestral S . La probabilidad del suceso complementario a E , $\bar{E}$, es $p(\bar{E}) = 1 - p(E)$

Teorema: Sean E_1 y E_2 sucesos de un espacio muestral S .
 $p(E_1 \cup E_2) = p(E_1) + p(E_2) - p(E_1 \cap E_2)$ (Prin. Inclusión-exclusión)

PROBABILIDAD DISCRETA

• Teoría de la probabilidad:

La definición de Laplace, $p(E) = \frac{|E|}{|S|}$, considera resultados igualmente probables, lo cual no es así en muchos experimentos.

Dado un espacio muestral de un experimento, con un número finito de posibles resultados. Generalización de Laplace: se asigna una probabilidad, $p(s)$, a cada resultado s , verificándose 2 condiciones.

$$1. 0 \leq p(s) \leq 1$$

$$2. \sum_{s \in S} p(s) = 1 \quad / \quad \sum_{i=1}^n p(s_i) = 1 \quad \text{para } n \text{ posible resultados}$$

La función p es llamada distribución de probabilidad. Permite modelar experimentos considerando resultados igualmente probables o no, escogiendo la $p(s)$ adecuada.

C.g:

Tirando al aire una moneda equilibrada, ¿qué probabilidad hay de caras y cruces?

$$p(c) = 1/2 \quad p(x) = 1/2$$

¿Y si la moneda estuviese sesgada por que salga el doble de caras que de cruces?

$$p(c) = 2p(x)$$

$$p(c) + p(x) = 1$$

$$2p(x) + p(x) = 1$$

$$p(x) = 1/3 \quad ; \quad p(c) = 2/3$$

NOTA: Sea S un conjunto de n elementos, la distribución uniforme asigna la probabilidad $1/n$ a cada elemento. Elegir un elemento en un espacio con distribución uniforme es seleccionar al azar.

Definición generalizada de la probabilidad de un suceso E : La probabilidad de un suceso E es la suma de las probabilidades de los resultados de E .

$$p(E) = \sum_{s \in E} p(s)$$

C.g: Si un dado tirado para que el 3 salga el doble de veces que cualquier otro número cuál es la probabilidad de que salga un número impar?

$$\text{Suceso } E = \{1, 3, 5\}$$

$$\sum_{i=1}^n p(x_i) = 1$$

$$p(1) = p(2) = p(4) = p(5) = p(6) = 1/6$$

$$p(3) = 2/6$$

$$p(E) = p(1) + p(3) + p(5) = 4/6$$

• Probabilidad condicionada.

La probabilidad condicionada es la probabilidad de que ocurra un suceso E , sabiendo que ya ha ocurrido un suceso F .

Sean E y F 2 sucesos tales que $p(F) > 0$, la probabilidad condicionada de E dado F se define $P(E/F) = \frac{P(E \cap F)}{P(F)} = \frac{P(F|E)P(E)}{P(F)}$ (Teorema de Bayes)

C.g: Dada una cadena de 4 bits generada al azar ¿cuál es la probabilidad de que contenga 2 ceros consecutivos, si sabemos que el primer bit es un 0?

Sea S los casos posibles empezando con 0, $p(F) = 8/16$. De los cuales cumple el requisito $E \cap F = \{0000, 0001, 0010, 0100\}$

Se saca de conteo

$$p(E/F) = \frac{5/16}{8/16} = \frac{5}{8}$$

- Probabilidad condicionada e Independencia.

2 sucesos son independientes $\Leftrightarrow p(E \cap F) = p(E) p(F)$

e.g. Sea E el suceso de que una cadena de 4 bits generada al azar comienza con 1, y F el suceso de que la cadena tenga un número par de 1s ¿son E y F independientes?

8 cadenas comienzan con 1, $p(E) = 8/16$. Tmb son 8 las cadenas que tienen n° par de 1s, $p(F) = 8/16$. $E \cap F = \{1111, 1100, 1010, 1001\}$???

$$p(E \cap F) = 4/16 = 1/4 = p(E) p(F) \rightarrow \text{Son sucesos independientes}$$

- Pruebas de Bernoulli

Se denomina a los experimentos con solo 2 posibles resultados, que suelen ser éxito o fracaso. E.g. Tirar 1 moneda: cara o cruz.

TEOREMA: La probabilidad de que haya K éxitos en n pruebas de Bernoulli independientes, con probabilidad de éxito p y de fracaso $q = 1-p$, es: $\binom{n}{K} p^K q^{n-K}$
 $\hookrightarrow p+q=1$

E.g.: Una moneda truca da con $p(C) = 2/3$. Suponiendo tiradas independientes, ¿cuál es la probabilidad de obtener 4 caras al tirar 7 veces la moneda?

Por la regla del producto hay $2^7 = 128$ posibilidades.

El n° de maneras a que los 7 resultados haya exactamente 4 caras es $\binom{7}{4}$. Luego, por el Tº: $\binom{7}{4} (2/3)^4 (1/3)^3$

- Variables aleatorias

NO es una variable que cambia al azar. Es una función que asigna un número real a cada resultado posible de un experimento

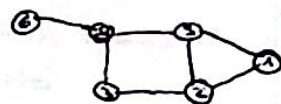
Una variable aleatoria es una función del espacio muestral de un experimento en el conjunto de los n° IR.

E.g.: Tiramos una moneda al aire 3 veces. $X(t)$ es la variable aleatoria que da el número de caras que aparecen en el resultado.

$$X(ccc) = 3; \quad X(ccx) = X(cxc) = X(xcc) = 2; \quad X(xxx) = 0 \dots$$

TEMA 4: Teoría de Grafos.

- Grafo simple: $G = (V, E)$: Conjunto no vacío de vértices/nodos (V), conectados entre algunos de ellos por enlaces denominados aristas (E), que pueden ser o no orientadas



- Grafo trivial: grafo con 0 aristas y 0/1 vértices ($V = \{\emptyset\}; E = \{\}; |V| = 1; |E| = 0$)

- Grafo dirigido: Cuando un conjunto de enlaces, arcos, es $E \subseteq V \times V$, entonces (V, E) es un par ordenado y el conjunto $G = (V, E)$ se llama grafo dirigido o digrafo y está formado por pares ordenados de vértices $\in V$ y arcos $\in E$.

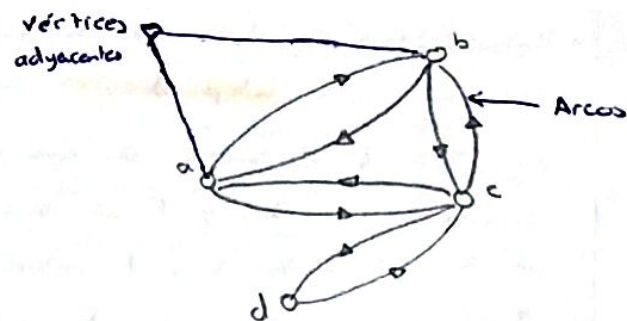
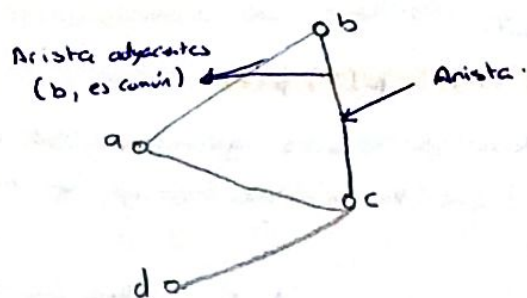
- $\hookrightarrow$ Orden: total de vértices
- $\hookrightarrow$ Tamaño: total de arcos.

Nota: El grafo sobre el que descansa un digrafo es un grafo subyacente.

PROBABILIDAD
DISCRETA

DEFINICIONES
Y
CONCEPTOS

DEFINICIONES Y CONCEPTOS



GRAFO (GRUPO SUBYACENTE)

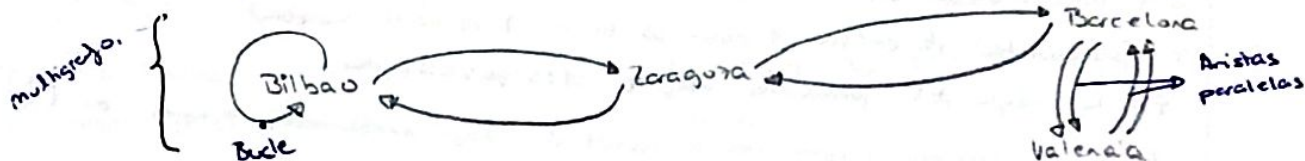
Pares no ordenados $\rightarrow E = \{ \{a,b\}, \{a,c\}, \{b,c\}, \{c,d\} \}$

DIGRAFO

Pares ordenados

$E = \{ (a,b), (a,c), (b,c), (c,d), (c,a), (c,b), (c,d), (d,c) \}$

- **Bucle:** Una arista lo forma cuando está asociada a 2 vértices iguales, es decir, una arista conecta un vértice consigo mismo.
- **Aristas / Arcos paralelos, aristas múltiples o multianistas:** Cuando 2 aristas son incidentes con los mismos vértices.
- **Multigrafo:** Si para $a, b \in V$ y $a \neq b$ $\exists$ 2 o + aristas $\{a,b\}$ (grafo) o (a,b) (digrafo).



NOTA: Cuando un grafo no posee ni **aristas paralelas**, ni **bucles** es un grafo simple.

- **Grafo con pesos / etiquetados:** es como se denomina a un grafo cuyas aristas están asociadas a un número real o etiqueta. (distancia, coste...)

Recorrido de Grafos.

↳ **Camino:** Secuencia alternada de vértices (v_i) y aristas (e_i) que tiene origen en v_1 y destino en v_{n+1} y que contiene 'n' aristas. La longitud de un camino es el número total de aristas 'n'. Si $n=0$, $\exists$ aristas $\rightarrow$ camino trivial.

↳ Un camino puede repetir tanto sus vértices como sus aristas.

↳ **Camino cerrado:** cuando $v_1 = v_{n+1}$ y $n \geq 1$, si no, el camino es abierto.

↳ **Camino simple:** aquel sin aristas repetidas, es decir, NO pasa 2 veces por la misma arista.

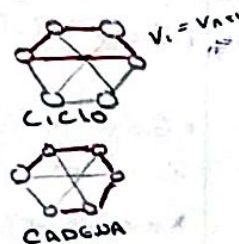
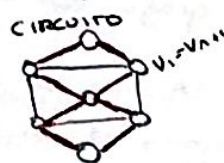
↳ **Camino elemental:** todo camino simple sin vértices repetidos, o sea, no pasa 2 veces ni por la misma arista ni por el mismo vértice.

- **Ciclo:** camino elemental cerrado
- **Cadena:** camino elemental abierto. *

• **Circuito:** camino simple cerrado, no arista repetida.

• Cualquier vértice único, puede considerarse como un camino, camino simple o camino elemental de longitud = 0.

* **NOTA:** una cadena de longitud n se denomina n-cadena, P_n .



• Características de circuitos y ciclos.

↳ Circuito: es necesario al menos una arista. Si hay solo 1 entonces Circuito = bucle.

↳ Ciclo: implica la presencia de al menos 3 aristas, así que la longitud de un ciclo ≥ 3 . Un n -ciclo = C_n ; 3-ciclo = C_3 . Todo ciclo es circuito.

$\triangle$ $\square$ $\pentagon$ $\hexagon$... Básicamente cualquier polígono.

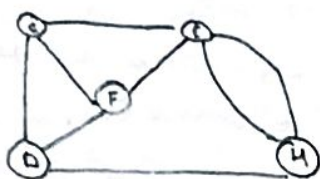
DEFINICIONES

CONCEPTOS

• Rueda (W_n): se obtiene a partir de un n -Ciclo. Basta con añadir un nuevo vértice tal que sea adyacente a los vértices primitivos del C_n (o sea lo conectamos a cada uno con nuevas aristas). $\triangle$ $\square$ $\pentagon$

• Grado de un vértice: Número de aristas que inciden en ese vértice.

• Grado de un grafo: suma de los grados de los vértices.



Grado (D) = 3

Grado (H) = 3

Grado (E) = 4

Grado (F) = 3

Grado (C) = 3

Grado del Grafo = 16

La nomenclatura viene dada por $\text{grad}(V)$, $\delta(V)$. Un vértice es de grado par/impar si el valor de su grado es par/impar.

• En el caso de un digrafo:

↳ Se denomina grado de entrada del vértice v al N° de arcos que llegan a ese vértice, $\text{gra}^-(v)$.

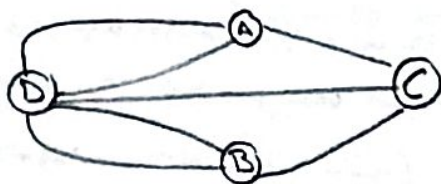
↳ El grado de salida del vértice v , se le llama al n° de arcos que salen de ese vértice, $\text{gra}^+(v)$.

Nota: un vértice con grado de entrada = 0 se llama **fuentes** y uno con grado de salida = 0 es un **sumidero**.

La **matriz de adyacencia** de un grafo de n vértices es una matriz $n \times n$ tal que a_{ij} representa el n° de aristas entre el vértice i y j .

↳ Si el grafo es no orientado, la matriz es simétrica,

↳ Si el grafo es simple, la matriz es booleana.



	A	B	C	D	Suma fila = grado
A	0	0	1	2	3
B	0	0	1	2	3
C	1	1	0	1	3
D	2	2	1	0	5
					$14 = \text{grado grafo}$

Teorema: La suma de los grados de todos los vértices de un grafo es igual al número de aristas. Ergo la suma de los grados de los vértices siempre es par.

Teorema: Sea A la matriz de adyacencia del grafo el elemento (i,j) de la matriz A^K indica el número de caminos de longitud K que lleva del vértice i al j . Si hacemos $M_K = A + A^2 + \dots + A^K$ conseguiremos el n° de caminos de longitud $\leq K$.

REPRESENTACIÓN

MATRIZ DE GRAFOS Y OTROS CONCEPTOS

• Matriz de incidencia.

Sea $G = (V, E)$ un grafo con $V = \{v_1, v_2, \dots, v_n\}$ como conjunto de vértices y $E = \{e_1, e_2, \dots, e_m\}$ como conjunto de aristas / Edos. La matriz de incidencia de G respecto a las ordenaciones de V y de E , es la matriz booleana, $n \times m$, $M = [m_{ij}]$, tal que:

$$m_{ij} = \begin{cases} 1, & \text{si la arista } e_j \text{ es incidente con } v_i \\ 0 & \text{en cualquier otro caso.} \end{cases}$$

REPRESENTACIÓN
MATRICIAL
DE
GRAFOS
Y
OTROS
CONCEPTOS

- Grafo conexo: si existe un camino elemental entre cualquier par de vértices. Un digrafo es conexo si su grafo subyacente también lo es.
- Grafo no conexo / disjunto: caso contrario al grafo conexo. Un grafo puede tener partes disjuntas llamadas componentes. El n° de estas se indica por $K(G)$

OTROS
GRAFOS

- Grafo regular de grado K / K -regular: grafo cuyos vértices son del mismo grado K , o sea, en cada vértice incide el mismo n° de aristas (K).

Lo Teorema: Sea $G = (V, E)$ un grafo regular de grado K , entonces: $K \cdot |V| = 2|E|$

- Grafo completo: grafo, no digrafo y sin bucles, con n vértices indicado por K_n , tal que cada vértice de K_n es adyacente con los $n-1$ vértices restantes, es decir, está conectado con todos los demás.

Lo Todos sus vértices de grado $n-1$

Lo El total de aristas equivale a $C_{n-1}^2 = \binom{n}{2}$



- Grafo Bipartido: $G = (V, E)$, aquel en el que $V = V_1 \cup V_2$ siendo $V_1 \cap V_2 = \emptyset$ y tal que cada lado de G es de la forma $\{a, b\}$, con $a \in V_1$ y $b \in V_2$. Es como si colocaras los vértices con 2 colores de forma que ningún vértice del mismo color estén conectados.

Lo Será completo, si además, cada vértice de V_1 está unido con cada vértice de V_2 . En este caso, si $|V_1| = m$ y $|V_2| = n$ el grafo se indicará como $K_{m,n}$



- Camino simple de Euler: "Sea $G = (V, E)$ un grafo o multigrafo sin vértices aislados", es un camino simple (sin aristas repetidas) que pase por todas las aristas de G solo 1 vez.

- Circuito de Euler: " " todo circuito (camino sin aristas repetidas y $V_i = V_{n+1}$) que pasa por todas las aristas de G solo una vez.

- Grafo Euleriano: aquel que posee un circuito euleriano.

Nota: Estos conceptos también se aplican a los digrafos.

GRAFOS
EULERIANOS

GRAFOS EULERIANOS

- Teorema de Euler: Sea $G = (V, E)$ un grafo o multigrafo sin vértices aislados. Entonces, G posee un circuito euleriano $\Leftrightarrow$ es **conexo** y sus **vértices son todos de grado par**.

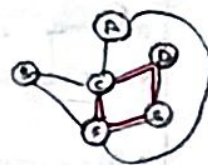
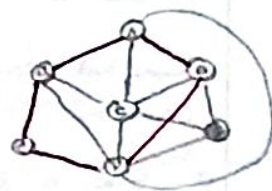
- Sea $G = (V, E)$ un grafo/multigrafo sin vértices aislados, se puede construir un camino de Euler, pero no un circuito euleriano, $\Leftrightarrow G$ es conexo y tiene exactamente **2 vértices de grado impar**. Entonces el camino simple de Euler puede comenzar y acabar en esos vértices. G se denomina grafo **Semi-Euleriano**.

TEOREMA: Sea $G = (V, E)$, un digrafo o multidigrafo sin vértices aislados. Entonces, G posee un circuito dirigido de Euler $\Leftrightarrow \text{gr}^-(v) = \text{gr}^+(v) \forall v \in V$.

- Construcción de grafos eulerianos

Algoritmo:

1. Tomamos un vértice cualquiera y formamos un circuito cualquiera C_1 (cuanto + grande mejor).
2. Obtenemos el subgrafo H_1 eliminando todas las aristas de C_1 , y quitando los vértices que quedan aislados. (todas las v_i de H_1 $\delta(v_i) = \text{par}$)
3. Iteramos hasta dar con un circuito euleriano.



En los grafos eulerianos hemos visto teoremas que establecían condiciones para encontrar caminos simples y circuitos que cubren todas las aristas, pero sin repetirse.

En los grafos Hamiltonianos será similar pero ahora considerándose todos los vértices del grafo sin repetir ninguno.

GRAFOS

HAMILTONIANOS

- Camino Hamiltoniano: Camino que pasa por todos los vértices una sola vez.
- Ciclo Hamiltoniano: Ciclo que pasa por todos los vértices una sola vez, empezando y acabando en el mismo vértice. *
- Grafo Hamiltoniano: grafo conexo que contiene un ciclo hamiltoniano.
 - ↳ El grafo debe ser conexo
 - ↳ No puede haber vértices de grado 1
 - ↳ No puede haber vértices de corte (q si los eliminamos junto a sus aristas se no queda un grafo no conexo)

Nota: K_n es hamiltoniano para $n \geq 3$.

- A diferencia de los eulerianos no hay un test sencillo, pero comprobarlo hay que comprobar todos los caminos posibles.

- Teorema de Dirac: Si G es un grafo simple y conexo con n vértices para $n \geq 3$ tal que todos los vértices tienen grado mayor o igual que $n/2$. Entonces, G es hamiltoniano (condición suficiente pero no necesaria).

GRAFOS

HAMILTONIANOS

- Teorema de Ore: Si G es un grafo simple y conexo con n vértices para $n \geq 3$ tal que $\text{grado}(u) + \text{grado}(v) \geq n$ para todos los pares de vértices no adyacentes u y v . Entonces G es hamiltoniano.

Lo contrario: sea $G = (V, E)$ un grafo sin bucles con $n \geq 2$ vértices. Entonces G tiene un camino de Hamilton, si $\text{grado}(v) \geq (n-1)/2$.

- Teorema: Sea $G = (V, E)$ un grafo simple con $n \geq 2$ vértices. Si se verifica que $\text{grado}(v) + \text{grado}(w) \geq n-1$, $\forall v, w \in V$, $v \neq w$, entonces G posee un camino de Hamilton. La existencia de bucles no afecta.

Volviendo a los grafos ponderados... $\rightarrow$ muy usado para optimización y N.

- Longitud camino ponderado: es la suma de los pesos de las aristas de ese camino.

En base a esto, muchos problemas son acerca de determinar un camino de longitud mínima entre 2 vértices de una red (grafo) (puede no ser único). Otro problema común es determinar el mismo camino pero que atraviese exactamente una vez cada vértice.

CAMINOS DE LONGITUD MÍNIMA

- Algoritmo de Dijkstra $\rightarrow$ en esto se basa Google maps

Lo Objetivo: hallar ruta + costo entre 2 vértices de un grafo ponderado.

1. El vértice inicial tiene distancia 0. A todos los demás se les pone ∞ (aún no sabemos llegar). Creamos un conjunto S vacío; $S =$ vértices ya "confirmados" con su distancia mínima definitiva.

2. Mientras hallas vértices sin procesar:

- 2.1 Escoger el vértice fuera de S con distancia + pequeña y lo añadimos a S confirmando que es la mínima posible.

- 2.2 Para cada vecino v de u comprobamos:

$\rightarrow$ La distancia actual que tenemos para v ; $L(v)$

$\rightarrow$ La distancia que obtendríamos pasando por u ; $L(u) + w(u, v)$

} Nos quedamos con la menor

3. Terminar cuando llegues al vértice final

Lo Algoritmo en pseudocódigo

1. Inicializa distancia: $L(a) = 0$, $L(\text{otros}) = \infty$

2. $S = \{ \}$

2. Mientras quedan vértices sin procesar:

- $u =$ vértice fuera de S con menor $L(u)$

- Añadir u a S .

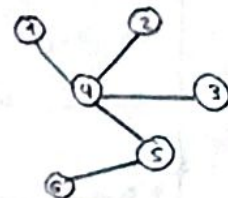
- Para cada vecino v de u :

$$L(v) = \min(L(v), L(u) + w(u, v))$$

4. Fin

TEMA 5: Árboles

- Árbol: grafo **conexo**, **no dirigido** y **sin ciclos** (acíclico).
Un árbol de n vértices tiene exactamente $n-1$ aristas.



TEOREMA: Grafo no dirigido es árbol $\iff$ entre cada par de vértices existe un único camino.

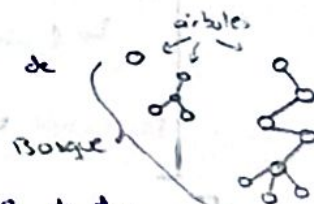
INTRODUCCIÓN A LOS ÁRBOLES

- Excentricidad:** máxima distancia de ese vértice a cualquier otro escogiendo el camino mínimo. Es la longitud del camino simple + largo desde ese vértice.
- Centro:** vértice (o conjunto de vértices) que tenga mínima excentricidad.



- Árbol degenerado:** $T = (V, E)$ que solo tiene 2 vértice y sin lados.
- Árbol maximal o generador:** Sea $G = (V, E)$, se denomina árbol maximal del grafo conexo G , un subgrafo maximal de G que tmb sea árbol. Un árbol maximal de G contiene todos los vértices V de G .
 $\hookrightarrow$ Se puede obtener en lugar de eliminando aristas (no + eficiente), añadiéndolas.
2 algoritmos basados en esta estrategia son: Búsqueda en Profundidad y Búsqueda en Anchura.

- Bosque:** grafo acíclico no conexo, en el cual cada una de sus componentes conexas es un árbol.



- Vértice raíz:** nodo del cual las aristas están orientadas alejándose de él. Un árbol con vértice raíz produce un grafo dirigido $\rightarrow$ árbol raíz. Se puede omitir el sentido de las aristas en un árbol raíz, pues sabemos que se aleja del vértice raíz. Cada elección distinta de la raíz original genera un árbol raíz distinto.

$\hookrightarrow$ Cada vértice v , distinto de la raíz, tiene como **padre** un único vértice u tal que hay una arista dirigida de u a v . Asimismo, v es **hijo** de u .

$\hookrightarrow$ Los vértices con el mismo padre son **hermanos**.

$\hookrightarrow$ Los **antecesores** de un vértice no raíz son todos los vértices que aparecen en el camino desde la raíz hasta ese vértice sin incluirlo a él.

$\hookrightarrow$ Los **descendientes** son aquellos para los que v es antecesor.

$\hookrightarrow$ Los vértices sin hijos se denominan **hojas**.

$\hookrightarrow$ Los vértices que tienen hijos se llaman **vértices internos**.

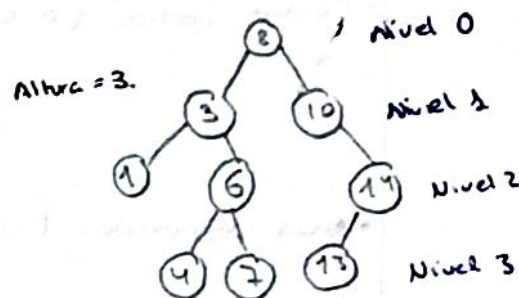
$\hookrightarrow$ La **RAÍZ** es un vértice interno si no es el único nodo del árbol. En ese caso sería una hoja.

INTRODUCCIÓN A LOS ÁRBOLES

- Árboles m -arios: si todos los vértices internos tienen a lo sumo m hijos. Si todo vértice interno tiene exactamente m hijos $\rightarrow$ es un árbol m -ario completo. Ej: árbol 2-ario; $m=2 \rightarrow$ árbol binario.
- Árbol ordenado con raíz: en el que los hijos de cada vértice interno están ordenados, y se ordenan de izq $\rightarrow$ der.
- Lo árbol con raíz en el hijo izq de un vértice = subárbol izquierdo.
- Lo árbol con raíz en el hijo der de un vértice = subárbol derecho.

- Nivel: el nivel de un vértice es la longitud del único camino desde la raíz del árbol a dicho vértice. El nivel del vértice raíz, por definición es 0.

- Altura: la altura de un árbol es la longitud del camino + largo desde la raíz a cualquier vértice (El máximo de los niveles de sus vértices).



TEOREMA: Un árbol de n vértices tiene $n-1$ aristas.

TEOREMA: Un árbol m -ario completo con i vértices internos tiene $n = mi + 1$ vértices.

TEOREMA: Un árbol m -ario completo con

• n vértices tiene: $i = (n-1)/(m-1)$ vértices internos y $l = [(m-1)n + 1]/m$ hojas.

• i vértices internos $n = mi + 1$ vértices y $l = (m-1)i + 1$ hojas

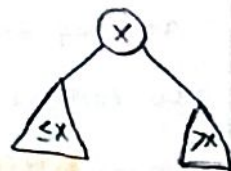
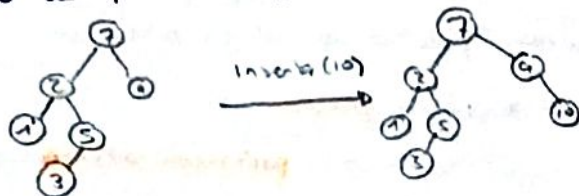
• l hojas tiene $n = (ml-1)/(m-1)$ vértices e $i = (l-1)/(m-1)$ vértices internos

TEOREMA: Un árbol m -ario de altura h , tiene a lo sumo, m^h hojas

Los árboles almacenan información / la codifican. Luego necesitamos estrategias para entender la estructura

- Árboles binarios de búsqueda (ABB): Los ABB son aquellos en que cada hijo de un vértice es hijo izq y hijo der. Tienen datos asociados a la raíz, y los datos del subárbol izq son menores que los del subárbol derecho. La búsqueda de un dato comienza comparándolo a la raíz, y según la comparación se decide por una de las ramas (mismo algoritmo para construirlos).

• Insertar: insertar un elemento en un ABB requiere buscar el lugar de inserción y crear una nueva hoja en caso de que no esté el elemento.

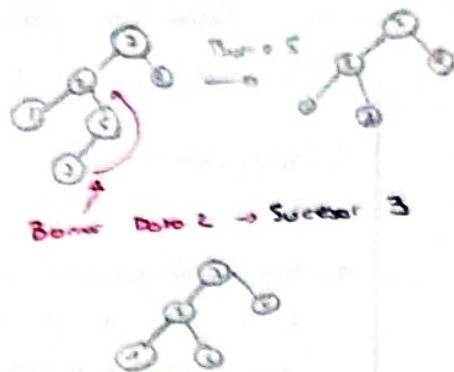


APLICACIONES DE ÁRBOLES

Lo Eliminar: eliminar un elemento implica primero buscarlo y, tras encontrarlo, eliminarlo. Si tiene un solo hijo, el padre cambia de hijo con el hijo único que tiene el nodo que se elimina (nieto pasa a ser hijo).



↳ **Borrar dato:** nos encontramos con un nodo con 2 hijos. Sustituimos el dato del nodo a borrar por su sucesor. El sucesor es el menor valor de todos aquellos que son mayores que el nodo a borrar → coger menor dato del subárbol derecho.



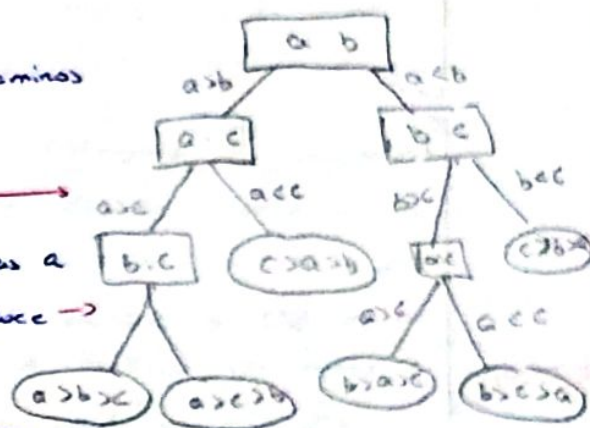
Nota: Si el árbol está balanceado el tiempo de búsqueda máxima se optimiza: $O(\log_2 N)$

- **Árboles de Decisión:** árbol con raíz en el que cada vértice interno corresponde a una decisión, con un subárbol a dichos vértices para cada resultado de la decisión.

Las posibles soluciones corresponden a los caminos desde la raíz hasta las hojas

↳ Ordenación de 3 elementos. $(a, b, c) \longrightarrow$

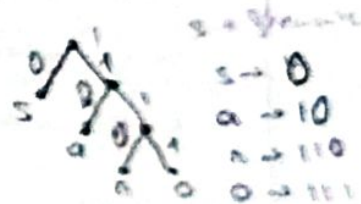
↳ Ordenación basada a comparaciones binarias a cada paso. El resultado de dicha comparación reduce $\rightarrow$ el n.º de ordenes posibles



2. Cada hoja representa una de las $n!$ permutaciones de los n elementos, $n=3$.

- **Código Instantáneo:** sistema de codificación donde ninguna palabra de código es prefijo de otra, es decir, que la cadena de bits no aparece al comienzo de la cadena de bits de otra palabra. Esto garantiza una decodificación casi inmediata. Otra ventaja de este sistema es que se puede asignar códigos + cortos a símbolos + frecuentes y + largos a menos frecuentes.

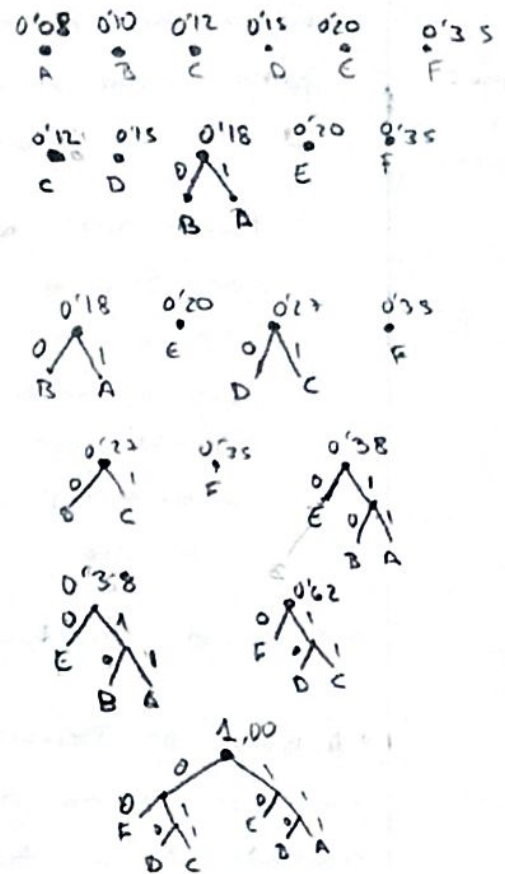
La Puntera representase por árbol binario. Los hijos corresponden a caracteres, los aristas izquierda y derecha con 0 y la derecha con 1. El código de un carácter es el camino desde la raíz hasta el hijo.



- **Código Huffman**: método de compresión que asigna a cada carácter una cadena de bits de longitud variable en función de la frecuencia de cada carácter. $\uparrow$ frecuente $\downarrow$ largo. Esto permite representar un mensaje usando el mínimo número total de bits entre todos los códigos de bits seleccionados posibles.

Algoritmo:

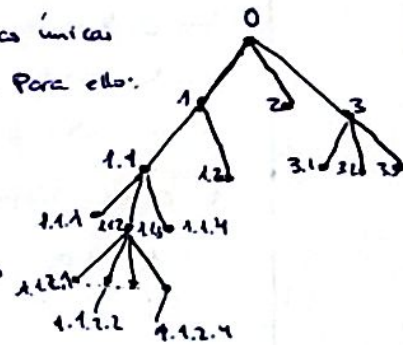
1. Cada carácter se representa como un árbol degenerado (una hoja) con su frecuencia asociada.
2. Este conjunto de nodos forma un bosque. Ordenamos cada nodo por su frecuencia de $\uparrow$ a $\downarrow$ de izq a dch.
3. Seleccionar los 2 nodos/árboles con frecuencia f $\uparrow$ $\downarrow$.
4. Se combinan. Se crea un nuevo nodo padre cuya f es la $+$ de los árboles seleccionados. El que tenga $\uparrow$ f $\rightarrow$ hijo izq; $\downarrow$ f $\rightarrow$ hijo dch.
5. Se elimina del bosque los dos árboles originales y se crea el nuevo.
6. Mientras $\exists$ $\uparrow$ de 1 árbol en el bosque repetir 3-5.
7. Una vez se tenga el árbol de Huffman (solo 1) se para.



- Sistema de Etiquetado Universal: procedimientos para visitar todos los vértices de un árbol ordenado con raíz. Los hijos de cada nodo están ordenados de izq a dch.

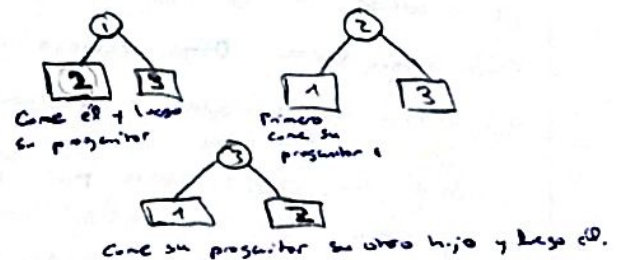
• El SEU o sistema de etiquetado canónico asigna etiquetas únicas a todas las vértices de un árbol ordenado con raíz. Para ello:

- Se etiqueta todos los vértices. El raíz con "0" y sus K hijos (nivel 1) con $1, 2, \dots, K$ de izq a dch.
- Para cada vértice del nivel n con etiqueta A , etiquetamos sus hijos K_n de izq a dch como: $A.1, A.2, \dots, A.K_n$.

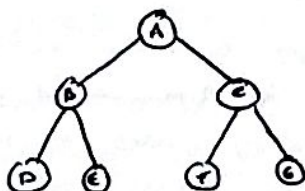


- Tipos de recorrido: hay 3 formas de visitar todos los nodos de un árbol ordenado con raíz.

- Pre-orden: La raíz se visita 1ª
- In-orden: La raíz se visita 2ª
- Post-orden: La raíz se visita última



C.g:



PRE-ORDEN

A B D E C F G

IN-ORDEN

D B E A F C G

POST-ORDEN

D E B F G C A

TEMA 6: Automatas y Computabilidad.

MODELOS DE COMPUTACIÓN (MC)

Hay 3 tipos de estructuras usadas en MC:

1. Gramáticas y lenguajes formales: reconocidos por las máquinas abstractas.
2. MEF (Máquinas de Estado Finito) con salida: correctores ortográficos, máquinas expendedoras...
3. Automatas finitos: Deterministas, No deterministas, De Pila.

Los Automatas Finito (AF) son subclases de máquinas secuenciales (MEF) pero sin salida. Su salida es Aceptar o no la palabra de entrada.

• Automata finito determinista

$A = \{I, S, O, f, h\}$ (sistemas)

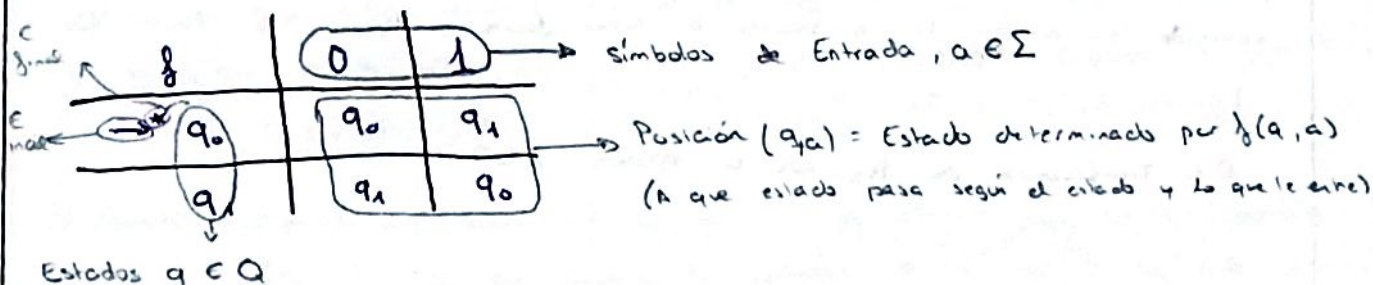
- ↳ I : Conjunto de entradas (Input)
- ↳ S : Conjunto de estados (state)
- ↳ O : Conjunto de salidas (Output)
- ↳ f : función de cambio de estado
- ↳ h : función de salida

$A = \{\Sigma, Q, f, q_0, F\}$

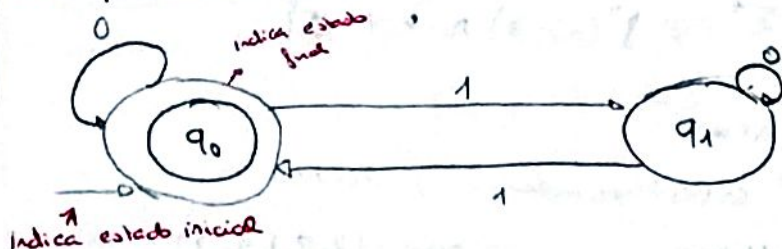
- ↳ Σ : Alfabeto de símbolos entrada
- ↳ Q : Conjunto finito de estados
- ↳ f : función de transición estados $f: Q \times \Sigma \rightarrow Q$
- ↳ q_0 : $q_0 \in Q$, estado inicial
- ↳ F : $F \subseteq Q$ Estados finales.

Note: f asigna a cada par (q, a) , un único estado de Q .

↳ Representación de un AFD (TT)



↳ Representación diagrama de transiciones (DT)



- Los estados son los nodos
- $\exists$ un arco etiquetado con "a" entre nodos q_i y q_j si: $f(q_i, a) = q_j$

↳ Universo del alfabeto (Σ^*): es el conjunto formado por todas las palabras que pueden formar con símbolos del alfabeto Σ , de cualquier longitud.

G.g. si $\Sigma = \{0, 1\}$ $\Sigma^* = \{\lambda, 0, 1, 00, 01, 10, 11, \dots\}$

✓ Contiene n° infinito de palabras

✓ La palabra vacía (λ) e a todas las veces.

↳ Lenguaje reconocido por un AFD: conjunto de palabras aceptadas por un AFD. Cadenas que transforman el q_0 en un estado final. $L_{AFD} = \{x \mid x \in \Sigma^* \vee f'(q_0, x) \in F\}$

CADA AFD reconoce un L $\begin{cases} \text{salida} = \text{aceptación si } x \in L \\ \text{salida} = \text{no aceptación si } x \notin L \end{cases}$

Note: un lenguaje regular es aquel aceptado por un AFD.

• Autómata finito no determinista (AFND)

Se define = que los AFD, a excepción de δ , pues en estas puede existir + de una transición para cada par (q, a) o Ninguna. Puede realizar transiciones entre estados sin recibir entradas (Transiciones λ). Ergo:

$$\delta: Q \times (\Sigma \cup \{\lambda\}) \rightarrow P(Q)$$

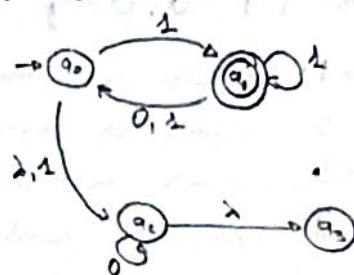
↳ Relación de transición - λ , T :

Relación entre pares de estados que mantienen una transición - λ . La relación es reflexiva; $\forall q \in Q \exists$ transición - λ con q , $\delta(q, \lambda) = q$, T consiste en un conjunto de tales pares de estados.

↳ Cierre transitivo de la relación T , T^* :

Incluye todos los estados que pueden alcanzarse desde otro usando cualquier número de transiciones λ , directas o encadenadas.

$$T^* = \{(q_0, q_0), (q_0, q_1), (q_0, q_2), (q_1, q_1), (q_2, q_2), (q_2, q_3), (q_1, q_3)\}$$



↳ Extensión a palabras:

$$\delta^*: Q \times \Sigma^* \rightarrow P(Q);$$

permite calcular el conjunto de estados alcanzables tras leer una palabra completa, teniendo en cuenta las transiciones λ antes, durante y después de leer los símbolos.

$$\delta^*(q, x) = \{p \mid x = a_1 a_2 \dots a_n \text{ y } q \xrightarrow{a_1} q_1 \xrightarrow{a_2} q_2 \dots \xrightarrow{a_n} p\}; \quad \delta^*(q, \lambda) = \{p \mid q \xrightarrow{\lambda} p\}$$

E.g: Transiciones de A_3 con la Palabra $\lambda \neq \emptyset$

$$\delta^*(q_0, \lambda) = \{q_0, q_1, q_2\}$$

↳ Lenguaje aceptado por un AFND:

$$L_{AFND} = \{x \mid x \in \Sigma^* \text{ y } \delta^*(q_0, x) \cap F \neq \emptyset\}$$

↳ Equivalencia de AFDs y AFNDs:

Para cada AFD $\exists$ un AFND equivalente y viceversa

→ AFD → AFND (AFD → caso particular del AFND)

$$1) \forall q \in Q, a \in \Sigma, |\delta(q, a)| = 1$$

$$2) \text{ No hay transiciones } \lambda (\forall q \in Q, \delta^*(q, \lambda) = \emptyset)$$

$$\rightarrow AFND = (Z, Q, \delta, q_0, F) \rightarrow AFD = (Z, Q', \delta', q_0, F')$$

$$\hookrightarrow Q' = P(Q)$$

$$\hookrightarrow q'_0 = \delta(q_0, \lambda)$$

$$\hookrightarrow F' = \{C \mid C \subseteq Q' \exists q \in C \text{ y } q \in F\}$$

$$\hookrightarrow \delta'(C, a) = \{C' \mid C' = \bigcup_{q \in C} \delta^*(q, a)\}$$

TEOREMAS
DE
KLEENE

Teorema de
análisis

Teorema de
síntesis

→ Todo lenguaje aceptado por un AF es regular

→ Todo LR (lenguaje regular) es aceptado por una AF

Los autómatas de pila son similares a los AF, pero con memoria auxiliar (pila). Gracias a esta reconocen lenguajes Tipo 2 (independientes del contexto).

Por tanto, es capaz de reconocer lenguajes del tipo $x^n y^n$, (1) , ... para lo cual se necesita memoria. Al leer "(" apilan un símbolo, al leer ")" despiden un símbolo.

NOTA: No pueden reconocer Gramática Tipo 3.

• Definición formal de un AP

$$AP = \{ \Sigma, \Gamma, Q, A_0, q_0, \delta, F \}$$

↳ Σ = Alfabeto de símbolos de entrada (cadena de entrada)

↳ Γ = Alfabeto de símbolos de la Pila ($\Sigma \cup \Gamma \neq \emptyset$)

↳ Q = Conjunto de Estados

↳ $A_0 \in \Gamma$ = símbolo inicial de la Pila

↳ $q_0 \in Q$ = Estado inicial

↳ $F \subseteq Q$ = Conjunto de estados finales

↳ δ = función de transición entre estados $\delta: Q \times (\Sigma \cup \{\lambda\}) \times \Gamma \rightarrow P(Q \times \Gamma^*)$

• Tipos de transiciones

↳ Transición Tipo 1 (con lectura): Si el AP está en estado q , lee símbolo a , y la cima de la pila es Z : $\delta(q, a, Z) = \{(q_i, z_i)\} \rightarrow$ Cambia de estado y reemplaza Z por la cadena z_i en la pila.

↳ Transición Tipo 2 (λ -transición): Igual que el anterior, pero sin leer entrada. $\delta(q, \lambda, Z) = \{(q_i, z_i)\}$. No se mueve la cabeza de lectura de la entrada.

• Descripción instantánea (DI)

Descripción sencilla de la configuración del AP en cada momento:

$$(q, x, z) \quad q \in Q, x \in \Sigma^*, z \in \Gamma^*$$

↳ q : estado actual ↳ x : parte de la entrada que queda por leer

↳ z : Contenido de la pila (cima a la ita)

Ej: AP, 1100, $\delta(q_0, 1, A) = \{(q_0, 1A)\}$

$$(q_0, 1100, A) \rightarrow (q_0, 100, 1A) \rightarrow \dots \rightarrow (q_1, \lambda, \lambda)$$

• Movimientos.

Si $(p, x) \in \delta(q, a, z)$ con $p, q \in Q$, $a \in \Sigma \cup \{\lambda\}$, $z \in \Gamma$, $x \in \Gamma^*$, entonces:

$$(q, ay, zx) \vdash (p, y, xX)$$

Ej: AP1 con palabra 1100

$$(q_0, 1100, A) \vdash (q_0, 100, 1A) \vdash (q_0, 00, 11A) \vdash (q_1, \lambda, \lambda)$$

AUTÓMATAS DE PILA

La Sucesión de movimientos: Existe una secuencia de 0 o + movimientos que permite llegar desde la descripción de la configuración:

$$(q_0, 1100, A) \vdash^* (q, \lambda, \lambda)$$

1-º Movimiento desde el inicio hasta el estado/situación final del AP.

AUTÓMATAS DE PILA

Esto indica: la palabra 1100 es aceptada; se llegó al estado final; la entrada se consumió; la pila quedó vacía.

• Autómatas de pila determinista (APD)

En ellos es aplicable en cada instante una y solo una transición, lo cual no elimina el indeterminismo. Un AP es determinista:

$$\hookrightarrow \forall q \in Q, A \in \Gamma \text{ si } |\delta(q, \lambda, A)| > 0 \Rightarrow \forall a \in \Sigma, \delta(q, a, A) = \emptyset \quad (1)$$

$$\hookrightarrow \forall q \in Q, A \in \Gamma, a \in \Sigma \cup \{\lambda\}, |\delta(q, a, A)| \leq 2 \quad (2)$$

(1) Si existe una transición λ para un símbolo de pila, entonces no puede haber transiciones con símbolos de entrada para ese mismo símbolo.

(2) Para cada combinación (q, a, A) , hay a lo sumo una transición.

• Lenguajes aceptados por un AP.

↳ Aceptación por vaciado de pila: una palabra es aceptada si $(q_0, x, A_0) \vdash^* (p, \lambda, \lambda)$ es decir, se consume toda la entrada y la pila queda vacía.

↳ Aceptación por estado final: si $(q_0, x, A_0) \vdash^* (p, \lambda, X)$, $p \in F$, es decir, se llega a un estado final y la pila puede tener cualquier contenido.

Las máquinas de Turing (MT) tienen el mayor poder computacional, + generales que los AF y AP. Capaces de aceptar los mismos lenguajes que pueden generar las gramáticas menos restrictivas $\rightarrow$ Gramáticas Estructuradas por frases.

NOTA: poseen mecanismo de control y flujo de entrada/cinta = AF, pero las MT pueden leer y escribir, mover su cabeza de lectura en 2 sentidos (I, D).

• Propiedades

↳ La cinta de lectura/escritura de una MT es teóricamente infinita con un extremo izquierdo delimitado.

↳ El conjunto de estados es finito

↳ Un MT necesita al menos 2 estados.

↳ Estado inicial $\rightarrow$ es la que la MT comienza los cálculos
↳ Estado Pausa $\rightarrow$ es la que terminan los cálculos
 $\neq$
Estado de aceptación (AF)

↳ Puede emplear su cinta como almacenamiento auxiliar

↳ $\exists$ 2 tipos de símbolos en la MT

Alfabeto de la máquina
Símbolo de cinta: puede incluir marcas al alfabeto

• Definición formal de las MT.

$$MT = \{\Gamma, \Sigma, \Delta, Q, q_0, \delta, F\}$$

↳ Γ : Alfabeto símbolo cinta

↳ $\Sigma \subset \Gamma$: Alfabeto símbolo entrada

↳ $\Delta \in \Gamma, \Delta \in \Sigma$: espacio en blanco

↳ Q : Conjunto estados

↳ $q_0 \in Q$: estado inicial

↳ δ : función de transición

↳ $F \subset Q$: Conjunto de estados finales

$$\delta: Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, D, R\}$$

MÁQUINAS DE TURING

Arquitectura de una MT

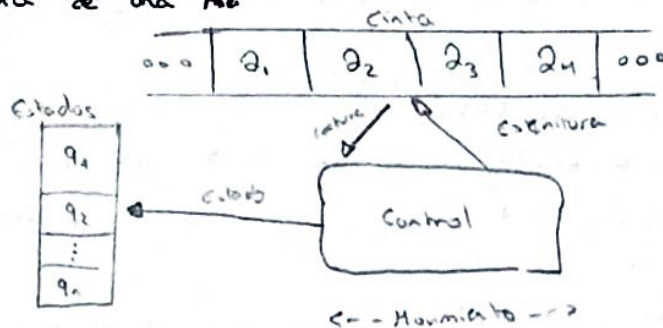


Tabla de Transiciones

f	1	0	b
P	q0D	p0I	r0D
q	q1D	q0D	p0I
r		r1D	s0P
s			

$$MT_1 = \{0, 1, \Delta\}, \{1\}, \Delta, \{p, q, r, s\}, \{p, q, r\}$$

Operaciones de las MT

Op. de escritura

Reemplazar un símbolo en la cinta con otro y cambiar de estado

Op. de Movimiento

Mover la cabeza una celda a la D o a la I y pasar a Nuevo Estado

La Representación funcional y sintáctica

$$f(q, a) = (p, x, m) \text{ o } p, x, m, q, p \in Q, a, x \in \Gamma, m \in \{I, D, P\}$$

NOTA: Las MT son representables por diagramas de transiciones, parecido a AF, pero a la vez se etiquetan con a/b.

- a: símbolo de cinta que debe Δ en celda actual para hacer la transición
- b: símbolo que escribirá en la celda actual si es de escritura la operación. También puede ser dirección de la transición (D, I, P) si la operación es de movimiento.

Lenguaje reconocido por MT

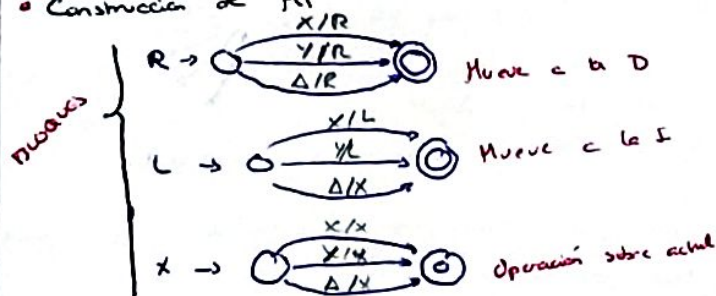
Una palabra es reconocida por una MT al llegar al estado final y estar en posición de parada. $L(MT) = \{x \mid (q_0, x) = (q, \Delta, P), x \in \Gamma^*, q \in F\}$

Las MT reconocen lenguajes que no pueden aceptar los AP: $\{x^n y^n z^n, n \in \mathbb{N}\}$

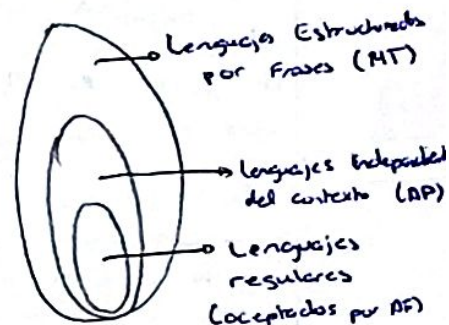
Las MT aceptan todos los lenguajes independientes del contexto.

Aceptan Gramáticas estructuradas por frases (GEF)

Construcción de MT



Con esto se genera MTs.



Diferentes tipos de máquinas de Turing.

Las MT pueden modificarse restringiendo o ampliando algunos aspectos (nº cintas, alfabeto de entrada, ...). No obstante el poder computacional siempre es el mismo.

MT con Alfabeto Binario (MTB)

Alfabeto cinta: $\Gamma = \{0, 1, \Delta\}$

Si k entrada original tiene m símbolos, se codifican usando n bits, ampliando: $2^{n-1} \leq m \leq 2^n$

MT con K cintas (MT multicinta)

Cada cinta tiene su propia cabeza de lectura/escritura. En cada paso la transición depende del estado actual y los K símbolos bajo las K cabezas. A transición afecta a 1 cinta.

TEOREMA: Toda MT de k cintas puede ser simulada por una MT de una sola cinta,
ergo $L(M) = L(M')$.

Las Máquinas de Turing No Deterministas (MTND)

- Puede haber + de 1 transición posible por un par (estado, símbolo).
- Si no hay transición aplicable $\rightarrow$ la máquina se detiene sin aceptar.
- Si hay varias la máquina elige 1.

Teorema: Por toda MTND existe una MT determinista equivalente (MTD), es decir, $L(MTND) = L(MTD)$.

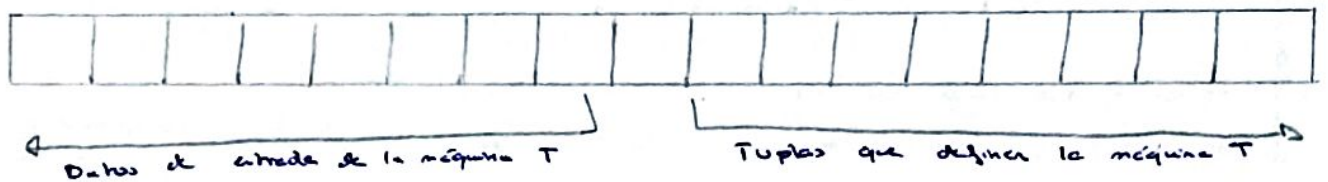
Nota: El poder computacional SIEMPRE es el mismo.

La MTU es una MT que recibe como entrada: la descripción codificada de otra MT (estado, transición...) y la cinta de entrada que esa MT debería procesar.

Básicamente simula paso a paso lo que haría esa MT original, produciendo el mismo resultado.

Nota: Equivalente teórico de un ordenador programable.

• Estructura de la cinta de la MTU.



La cinta es infinita en ambos sentidos y se divide en 2 partes separadas por el símbolo \$

Nota: a la izq, incluye un símbolo especial * que marca la posición de la cabeza lectora.

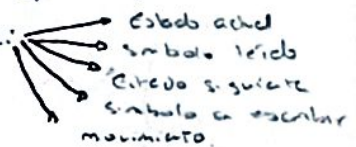
• Codificación de MT simulada.

La Codificación cinta

- Se copia tal cual la cinta
- Se marca la posición de la cabeza con *
- El bit bajo la cabeza se copia tal como está

La codificación de la máquina

- Cada transición se convierte en un registro
- Cada registro está separado por #
- Cada registro contiene:



• Computabilidad: qué problemas pueden resolverse mediante un algoritmo ejecutado por un modelo como una MT. Básicamente, computabilidad = qué puede calcularse.

• Complejidad: cuántos recursos necesita un algoritmo para resolver un problema, es decir, complejidad = cuán difícil/costoso es calcularlo.

MÁQUINAS
DE
TURING

MÁQUINA
DE
TURING
UNIVERSAL

COMPLEJIDAD
Y
COMPUTABILIDAD